Norges miljø- og biovitenskapelige universitet

TEL200 – Introduction to Robotics

David A. Anisi

Chapter 7: Robot manipulator kinematics

TEL200 Introduction to Robotics

Change of schedule from next week:

• **Project/lab:**	Tuesdays	14.15-18.00	TF4-102
	~~Thursdays~~	~~12.15-14.00~~	TF4-102
	Fridays	8.15-10.00	TF4-102

Check time-edit for updated schedule!

Agenda

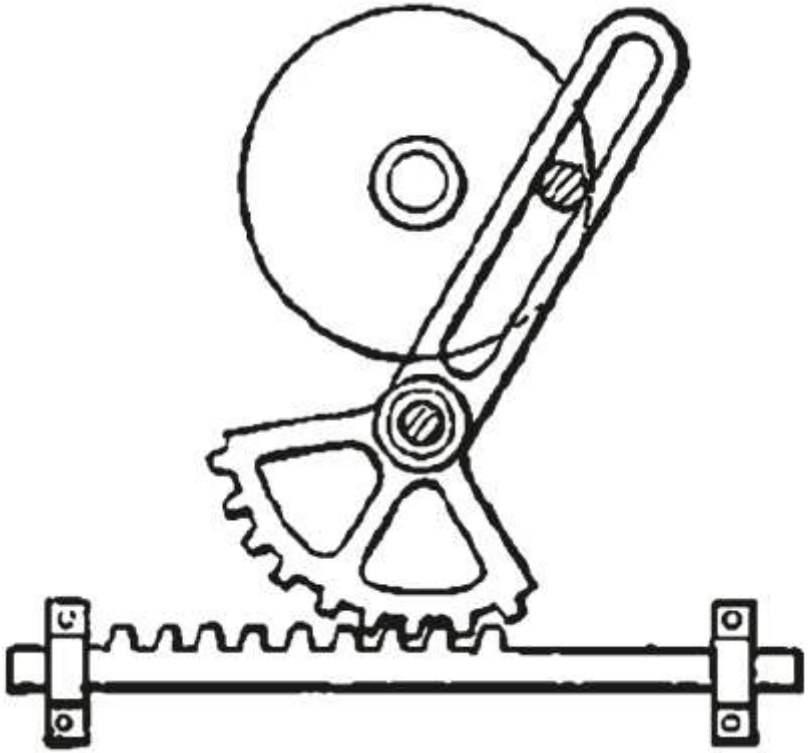
7.1 Forward Kinematics

7.2 Inverse Kinematics

7.3 Trajectories

Kinematics

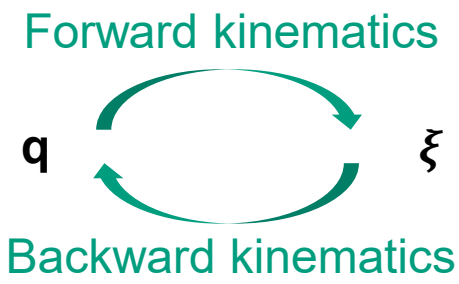
- From the Greek word for motion
- A branch of mechanics that studies the motion of a body, or system of bodies
- Concerned with positions (and angles) and velocities (translational and angular)
- Not concerned with mass, forces or moments (that's Dynamics, Ch. 9)



Kinematics

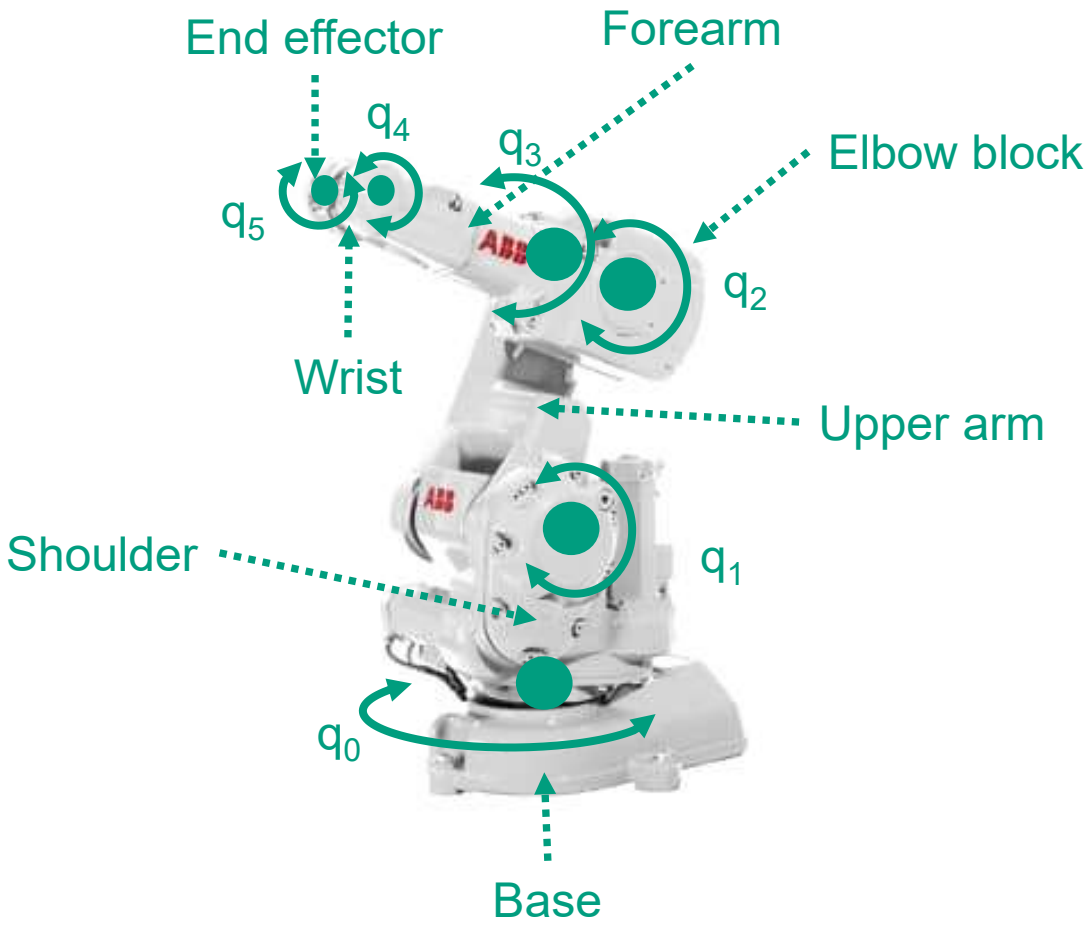
The relationship and mapping between configuration (e.g., joint coordinates $\mathbf{q} = [q_1, \dots, q_N]$)

and end-effector pose (e.g., Cartesian: $\xi = [x, y, z, \Phi_1, \Phi_2, \Phi_3]$)

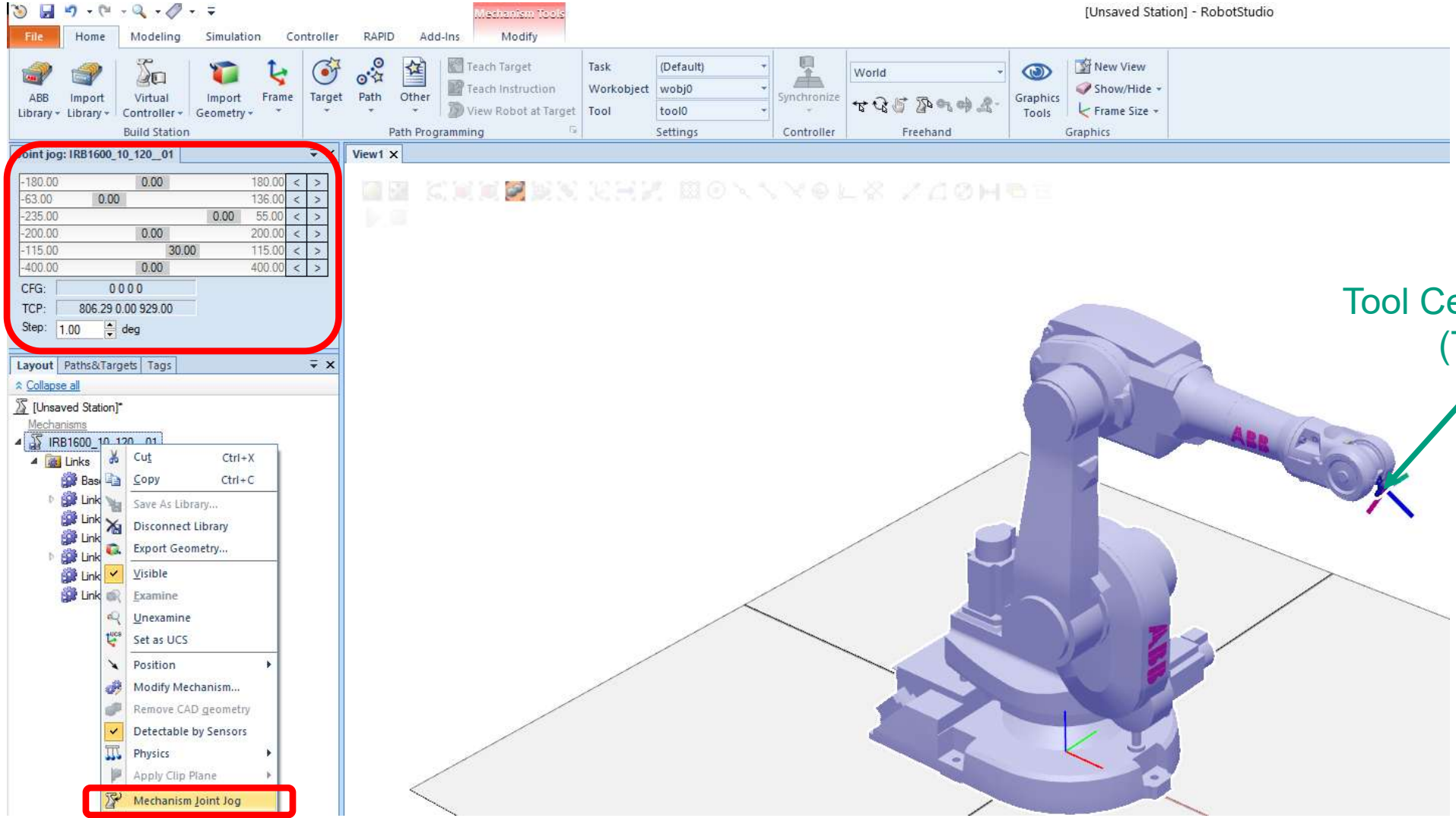


Some robot manipulator terminology

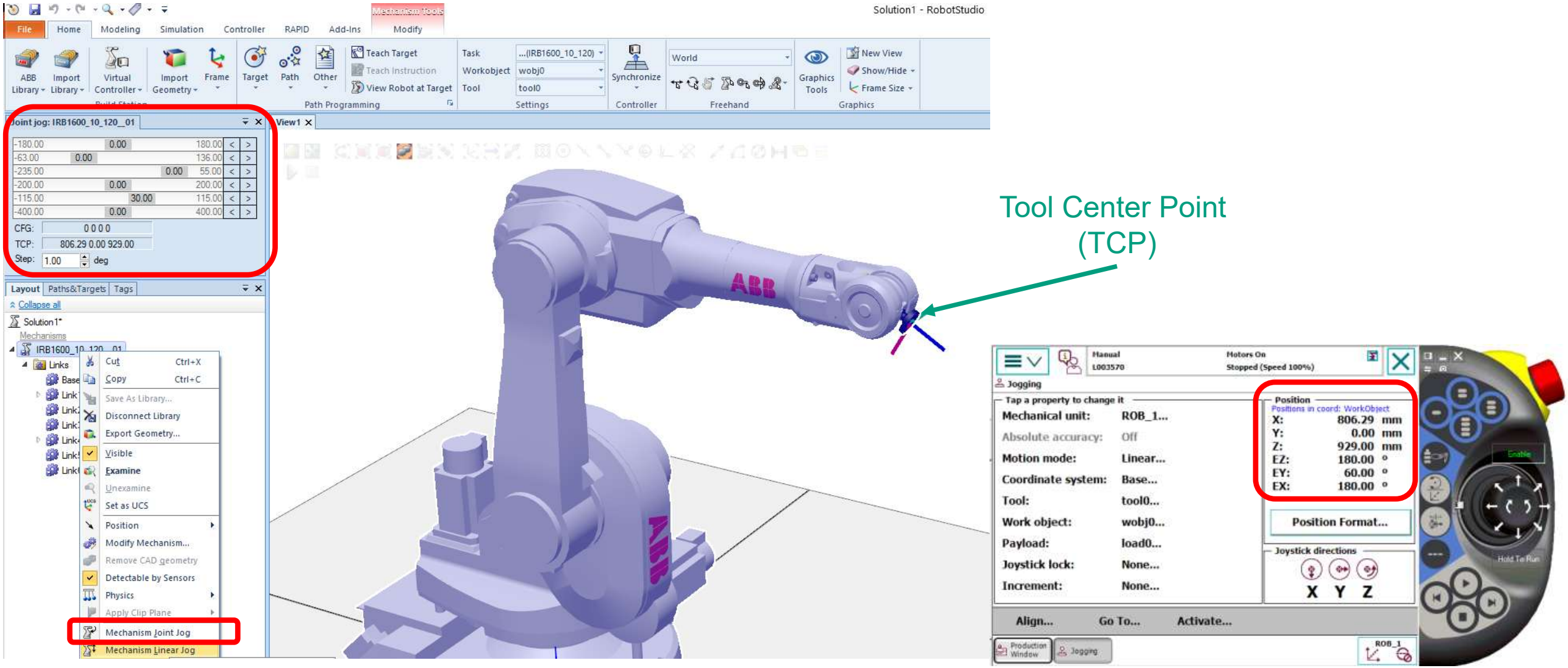
- **Base** is the link of the manipulator which is usually connected to the ground/mobile unit
- **Joint** is a part of the robot body which allows controlled or free relative motion of two links. Joint q_i connects $Link_i$ and $Link_{i+1}$
- **Link** is the rigid part of the robot body (upper arm, forearm, etc.)
- **End effecter** is the link of the manipulator which is used to hold the tools (gripper, spray/welding gun, etc.)



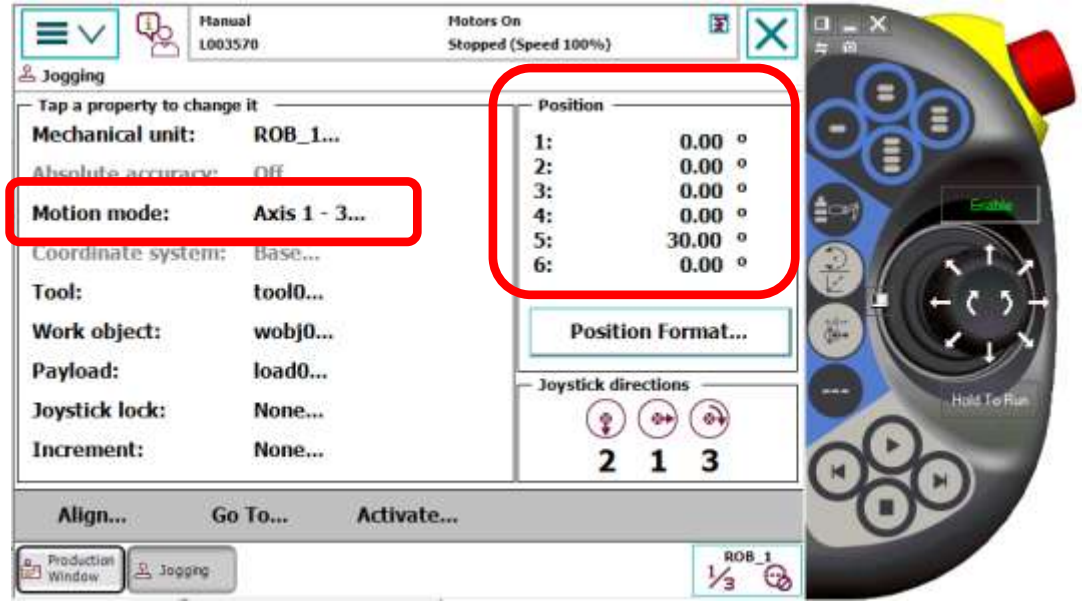
How do we go between joint-values and TCP?

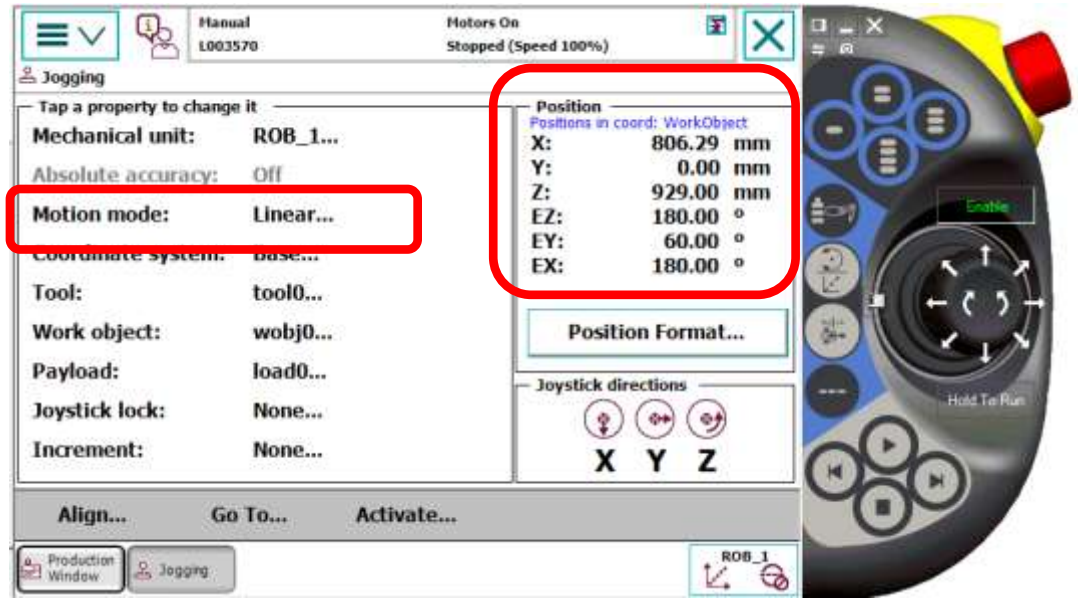


How do we go between joint-values and pose? vice verca?

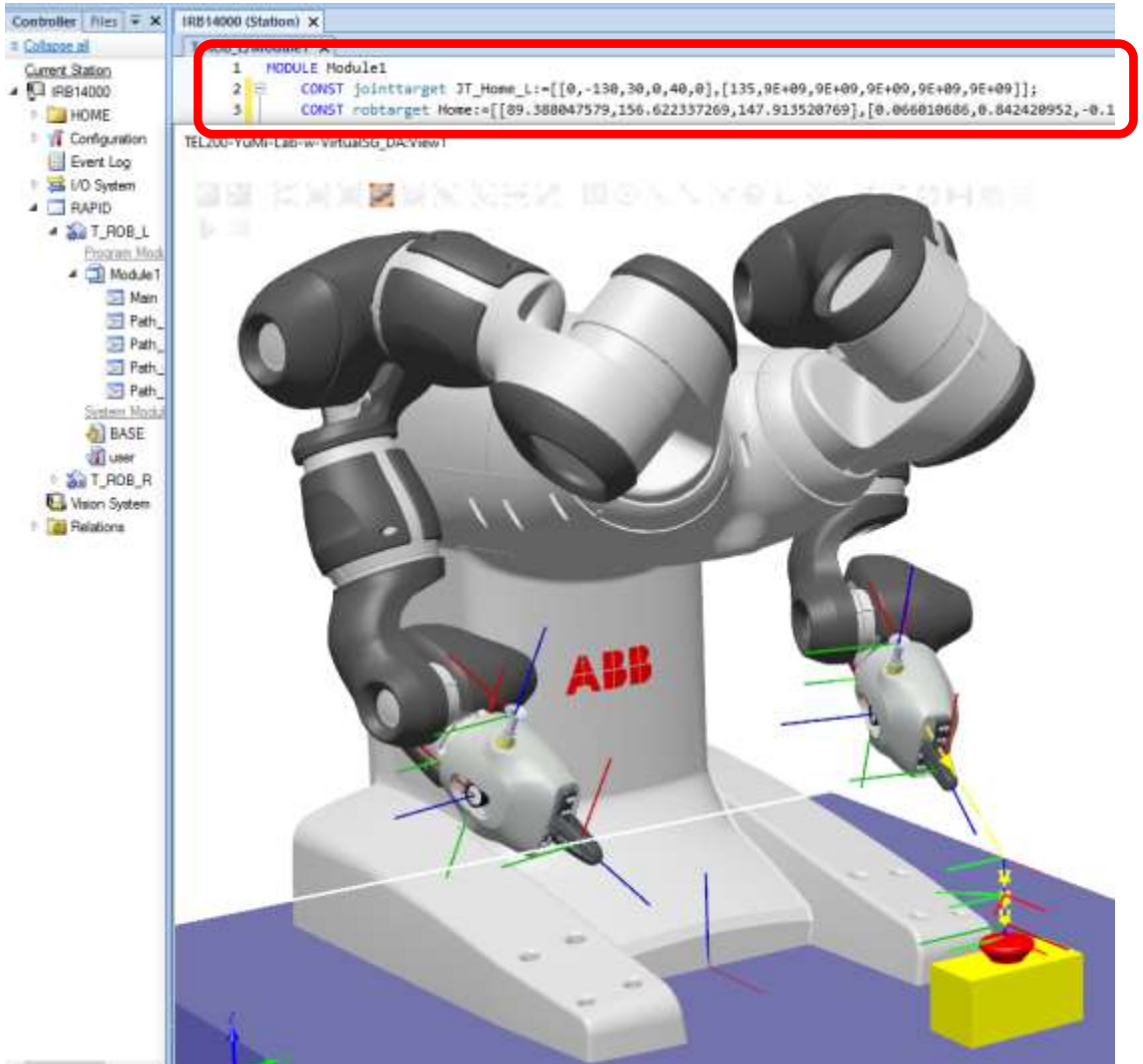


How do we go between linear- and axis-wise jogging?





How do we go between robtarget and jointtarget?



How do we go between robtarget and jointtarget?

Arguments

```
CalcRobT( Joint_target Tool [\WObj] )
```

Example 1

```
VAR robtarget p1;
CONST jointtarget jointpos1 := [...];

p1 := CalcRobT(jointpos1, tool1 \WObj:=wobj1);
```

Arguments

```
CalcJointT ( [\UseCurWObjPos] Rob_target Tool [\WObj]
    [\ErrorNumber])
```

Example 1

```
VAR jointtarget jointpos1;
CONST robtarget p1 := [...];
jointpos1 := CalcJointT(p1, tool1 \WObj:=wobj1);
```

Arm Types

- Arm-type robots or ***robot manipulators*** are a very common type of robot.

IRB 360

Payload: up to 8 kg
Reach: 0,8 – 1,6 m

Parallel/Delta robot

IRB 910SC

Payload: 6 kg
Reach: 0.45, 0.55, 0.65 m

SCARA robot

Arm Types

- Arm-type robots or ***robot manipulators*** are a very common type of robot.

IRB 360

- Food handling
- Picking and packing tasks

Parallel/Delta robot

IRB 910SC

- Material handling
- Small parts assembly
- Inspection

SCARA robot

Arm Types

- Arm-type robots or ***robot manipulators*** are a very common type of robot.

IRB 6620LX

Payload: 150 kg
Reach: 1.8 – 33.0 m (L)
2.5 – 4.0 m (H)

Gantry robot

IRB 660

Payload: 250 kg
Reach: 3.15 m

Articulated robot

Arm Types

- Arm-type robots or ***robot manipulators*** are a very common type of robot.

IRB 6620LX

- Machine tending
- Heavy arc welding
- Grinding
- Painting

IRB 660

- Material handling
- Palletizing

Arm Types

- Arm-type robots or ***robot manipulators*** are a very common type of robot.

YuMi – IRB 14000

Dual-arm robot

YuMi – IRB 14050

Single-arm robot

Inherently safe design: rounded edges, lightweight, power/force limitation, force/torque (F/T) sensors;

Arm Types

- Arm-type robots or ***robot manipulators*** are a very common type of robot.

YuMi – IRB 14000

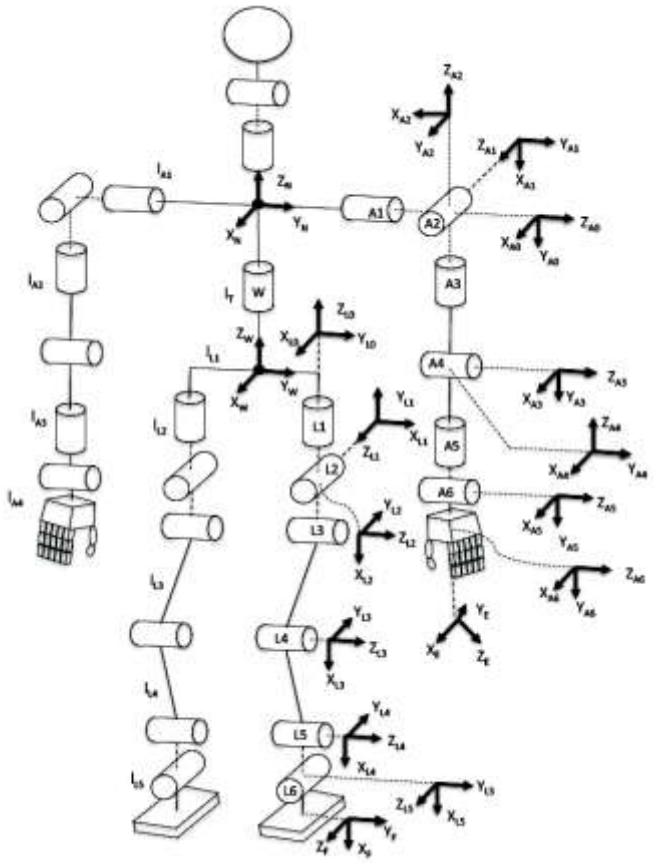
YuMi – IRB 14050

Human-robot collaboration:

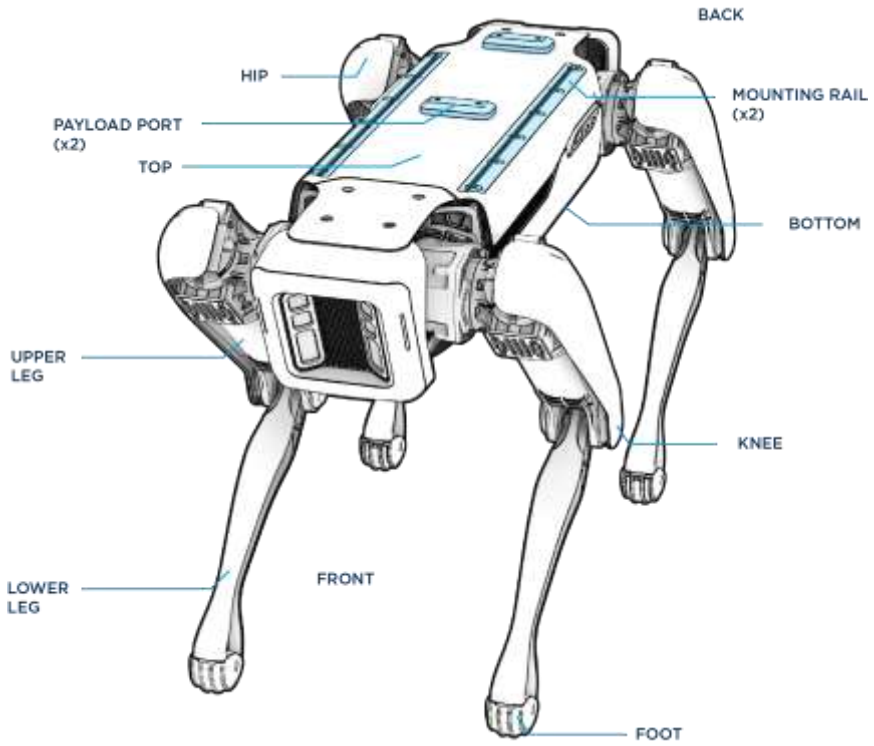
- Assembly
- Pharmaceutical
- Healthcare

Manipulators with branching chains

Atlas



HUBO2

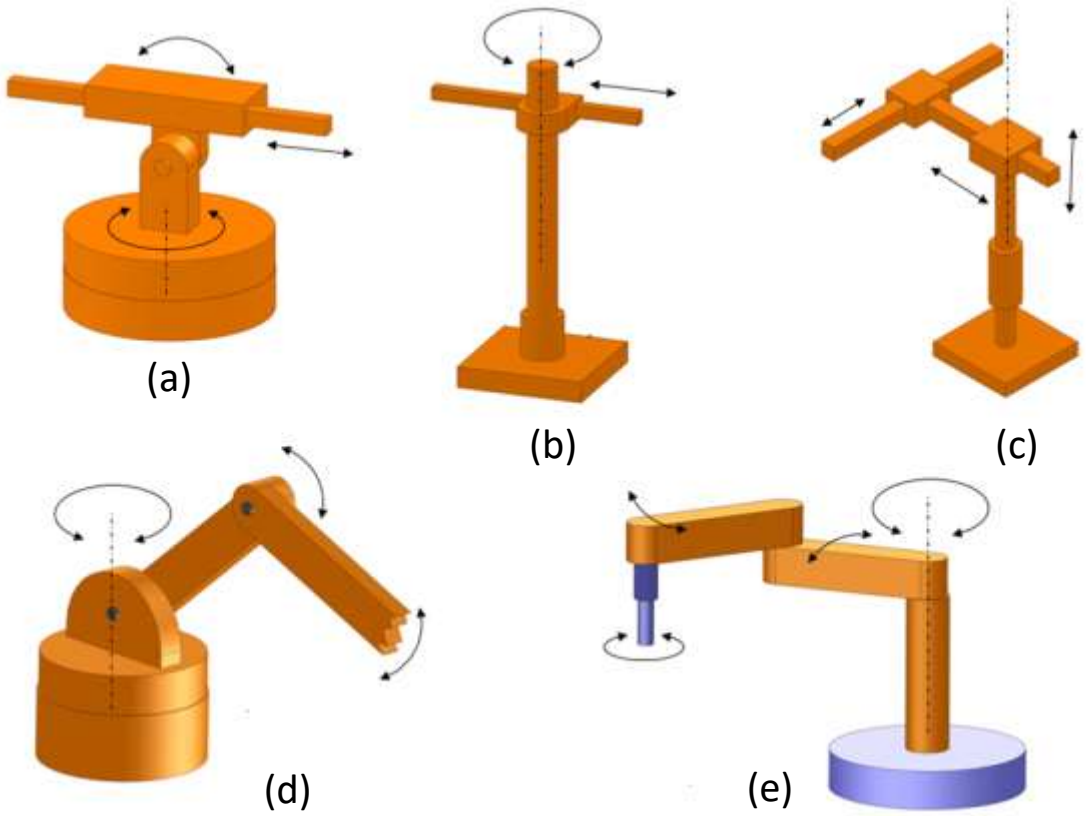


Spot

Industrial robots

- Classification: ***serial*** mechanical structure;
    a. Spherical (1%);
    b. Cylindrical (12%);
    c. Cartesian (20%) ;
    d. Articulated (59%);
    e. SCARA (8%).

Source IFR (2019): Percentage of installed robot manipulators worldwide.

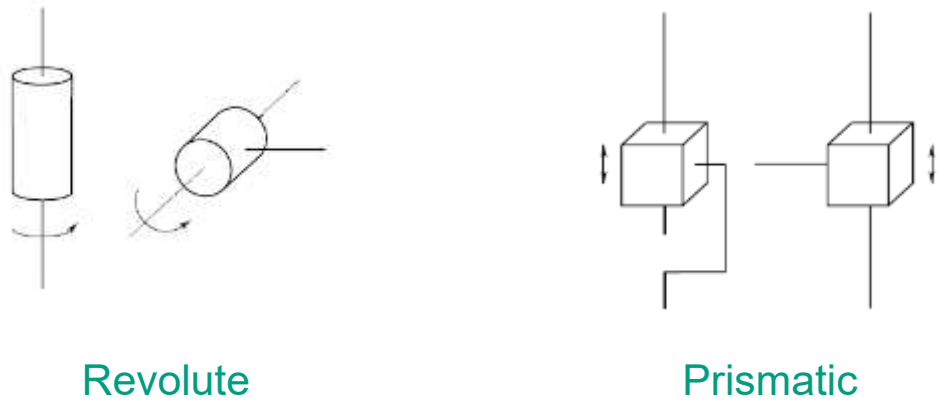


Introduction

- A robot arm, more formally a ***serial-link manipulator***, comprises a chain of rigid links and joints;

Introduction

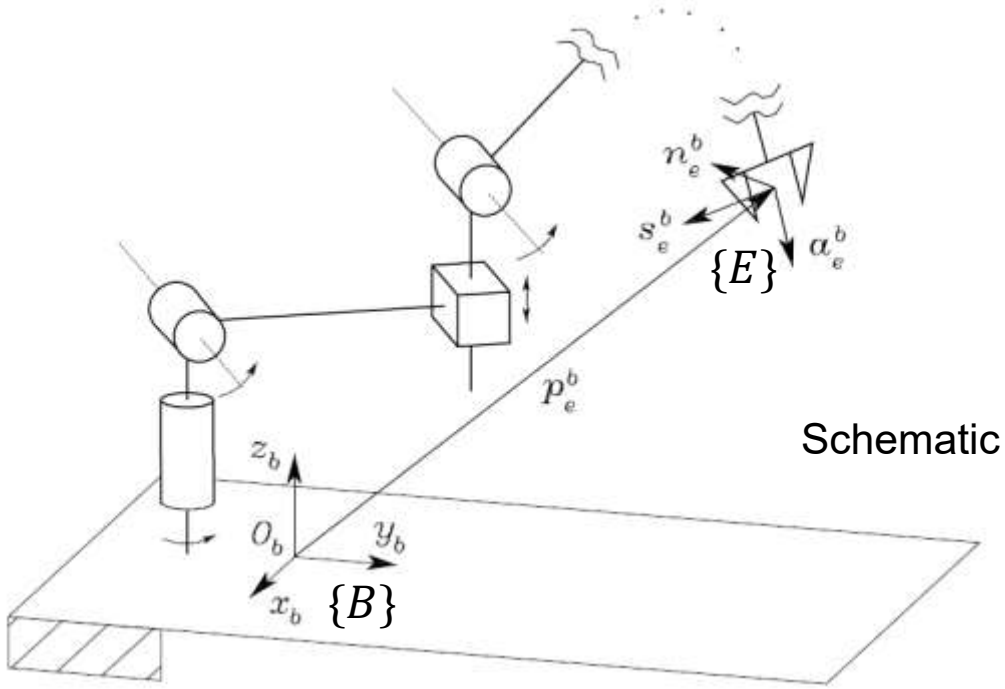
- A robot arm, more formally a ***serial-link manipulator***, comprises a chain of rigid links and joints;
- Each joint has *one D*egree of Freedom (DoF), either translational (a sliding or prismatic joint) or rotational (a revolute joint).



Schematic representation of robot joints [2].

Introduction

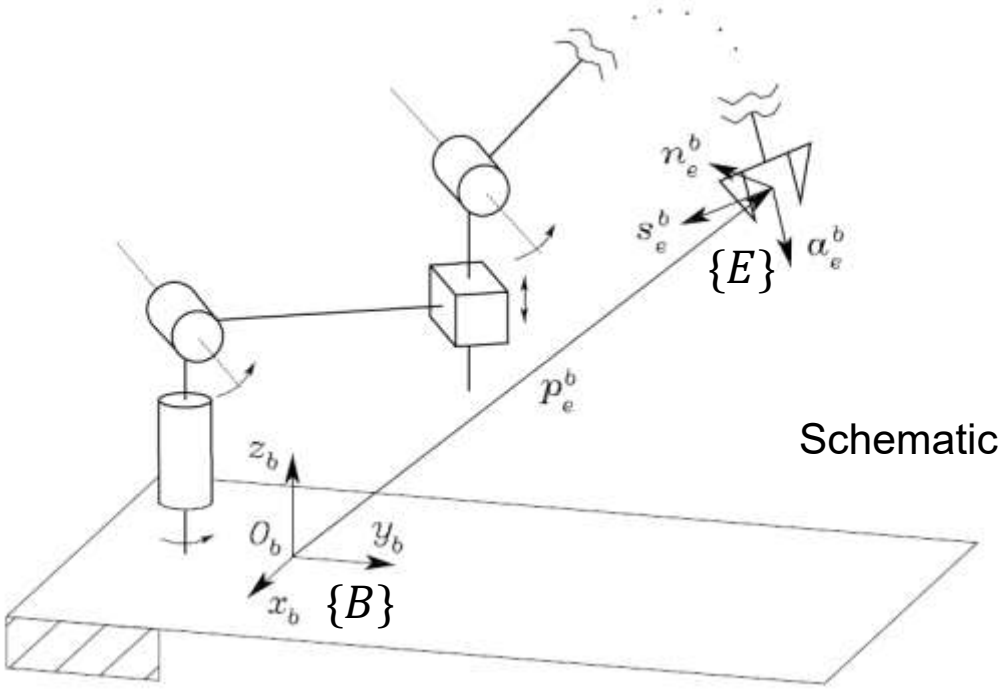
- Motion of a given ***joint*** changes the relative pose of the ***links*** that it connects;
- One end of the chain, the base, is generally **fixed** and the other end is **free** to move in space and holds the tool or *end-effector* that does the useful work.



Schematic representation of a robot arm [2].

Introduction

- Motion of a given ***joint*** changes the relative pose of the ***links*** that it connects;
- One *end* of the chain, the base, is generally **fixed** and the other end is **free** to move in space and holds the tool or *end-effector* that does the useful work.



Position: $p_e^b = [\, x_e^b \;\; y_e^b \;\; z_e^b \,]$

Orientation: $R_e^b = [n_e^b \;\; s_e^b \;\; a_e^b]$

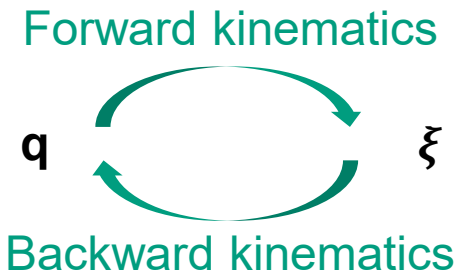
Schematic representation of a robot arm [2].

Forward Kinematics (Ch. 7.1)

- ***Forward kinematics*** is the mapping from joint coordinates $\boldsymbol{q}$, or robot configuration, to end-effector pose $\boldsymbol{\xi_E}$ given by:

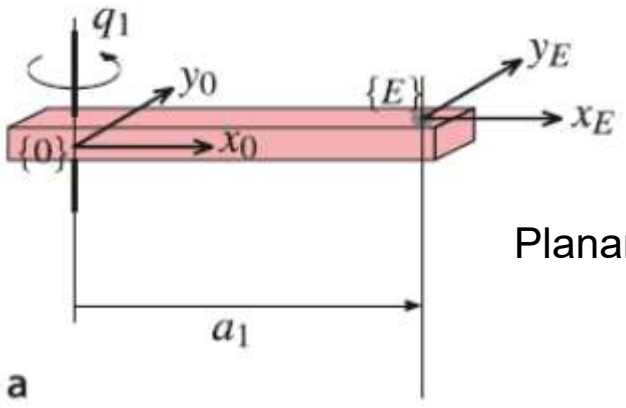
$$\boldsymbol{\xi_E} = \mathcal{K}(\boldsymbol{q}) \tag{1}$$

where $\mathcal{K}(\cdot)$ is - in general - a ***nonlinear function***



2-Dimensional (Planar) Robotic Arms

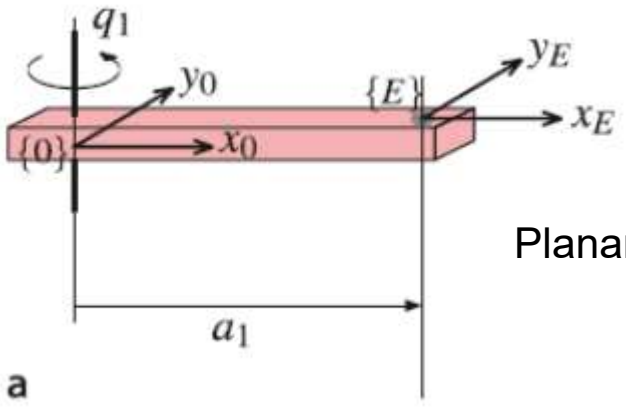
- Consider this simple robot arm, which has a single rotational joint.



Planar arm with one rotational joint.

2-Dimensional (Planar) Robotic Arms

- Consider this simple robot arm, which has a single rotational joint.



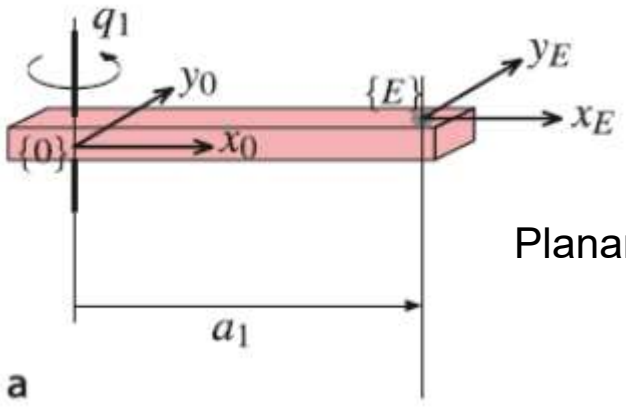
Planar arm with one rotational joint.

- We can describe the pose of its end-effector – frame $\{E\}$ – by a sequence of relative poses:

…a ***rotation*** about the joint axis q_1 and then a ***translation*** by a_1 along the rotated x-axis;

2-Dimensional (Planar) Robotic Arms

- Consider this simple robot arm, which has a single rotational joint.



Planar arm with one rotational joint.

- We can describe the pose of its end-effector – frame $\{E\}$ – by a sequence of relative poses:

$$\xi_E(\boldsymbol{q}) = \mathscr{R}_z(q_1) \oplus \mathscr{T}_x(a_1) \tag{2}$$

2-Dimensional (Planar) Robotic Arms

- Then, the ***forward kinematics map*** is given by:

$$\xi(q) = \begin{bmatrix} \cos q_1 & -\sin q_1 & a_1 \cos q_1 \\ \sin q_1 & \cos q_1 & a_1 \sin q_1 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

2-Dimensional (Planar) Robotic Arms

- Then, the ***forward kinematics map*** is given by:

$$\xi(q) = \begin{bmatrix} \cos q_1 & -\sin q_1 & a_1 \cos q_1 \\ \sin q_1 & \cos q_1 & a_1 \sin q_1 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

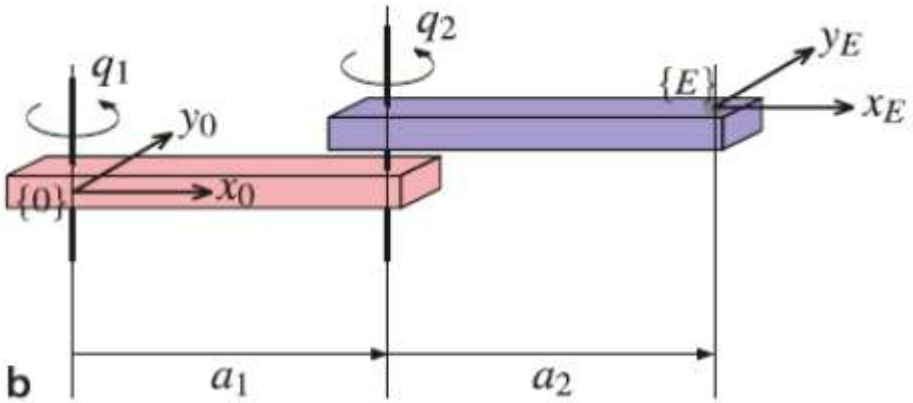
- The forward kinematics, for a ***particular*** value of $q_1 = 30 \text{ deg}$ and for the case $a_1 = 1 \text{ m}$, yields:

$$\xi(30) = \begin{bmatrix} 0.8660 & -0.5000 & 0.8600 \\ 0.5000 & 0.8660 & 0.5000 \\ 0 & 0 & 1 \end{bmatrix} \tag{4}$$

which is an **SE**(2) homogeneous transformation matrix representing the pose of the end-effector – coordinate frame $\{E\}$.

2-Dimensional (Planar) Robotic Arms

- Consider now a robot arm with ***two joints***



Planar arm with two rotational joints.

- The pose of the ***end-effector*** is given by:

$$\xi_E(\boldsymbol{q}) = \underbrace{\mathscr{R}_z(q_1)}_{\text{joint 1}} \oplus \underbrace{\mathscr{T}_x(a_1)}_{\text{link 1}} \oplus \underbrace{\mathscr{R}_z(q_2)}_{\text{joint 2}} \oplus \underbrace{\mathscr{T}_x(a_2)}_{\text{link 2}} \tag{5}$$

2-Dimensional (Planar) Robotic Arms

- Then, the ***forward kinematics map*** is given by:

$$\xi(q) = \begin{bmatrix} \cos q_{12} & -\sin q_{12} & a_1 \cos q_1 + a_2 \cos q_{12} \\ \sin q_{12} & \cos q_{12} & a_1 \sin q_1 + a_2 \sin q_{12} \\ 0 & 0 & 1 \end{bmatrix} \qquad q_{12} = q_1 + q_2 \tag{6}$$

2-Dimensional (Planar) Robotic Arms

- Then, the ***forward kinematics map*** is given by:

$$\xi(q) = \begin{bmatrix} \cos q_{12} & -\sin q_{12} & a_1 \cos q_1 + a_2 \cos q_{12} \\ \sin q_{12} & \cos q_{12} & a_1 \sin q_1 + a_2 \sin q_{12} \\ 0 & 0 & 1 \end{bmatrix} \qquad q_{12} = q_1 + q_2 \tag{6}$$

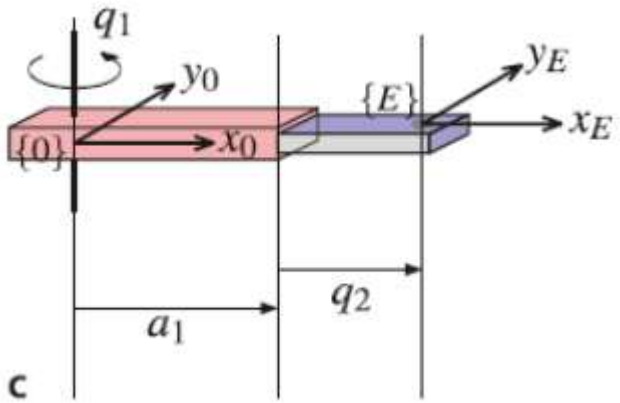
- The forward kinematics, for a ***particular*** value of $q_1 = 30\ \mathrm{deg}$ and $q_2 = 40\ \mathrm{deg}$, and for the case $a_1 = a_2 = 1\ \mathrm{m}$, yields:

$$\xi(30, 40) = \begin{bmatrix} 0.3420 & -0.9397 & 1.208 \\ 0.9397 & 0.3420 & 1.440 \\ 0 & 0 & 1 \end{bmatrix} \tag{7}$$

which is an **SE**(2) homogeneous transformation matrix representing the pose of the end-effector – coordinate frame $\{E\}$.

2-Dimensional (Planar) Robotic Arms

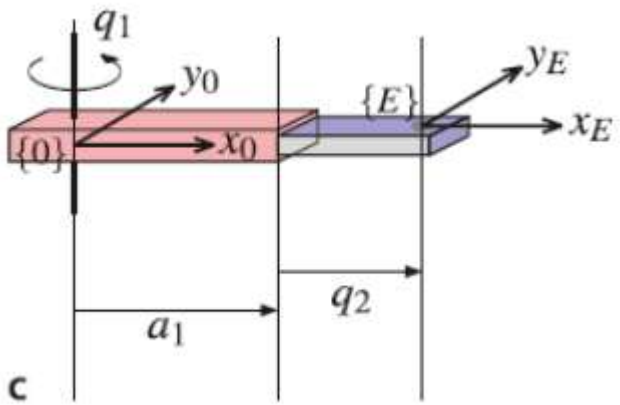
- So far, we have ***only*** considered revolute joints, but we could use a prismatic joint instead



Planar arm with two joints: one rotational and one prismatic.

2-Dimensional (Planar) Robotic Arms

- So far, we have ***only*** considered revolute joints, but we could use a prismatic joint instead



Planar arm with two joints: one rotational and one prismatic.

- The pose of the ***end-effector*** is given by:

$$\xi_E(\boldsymbol{q}) = \underbrace{\mathscr{R}_z(q_1)}_{\text{joint 1}} \oplus \underbrace{\mathscr{T}_x(a_1)}_{\text{link 1}} \oplus \underbrace{\mathscr{T}_x(q_2)}_{\text{joint 2}} \tag{8}$$

2-Dimensional (Planar) Robotic Arms

- We can easily add a ***third*** (revolute) joint

$$\xi_e(\boldsymbol{q}) = \underbrace{\mathscr{R}_z(q_1)}_{\text{joint 1}} \oplus \underbrace{\mathscr{T}_x(a_1)}_{\text{link 1}} \oplus \underbrace{\mathscr{R}_z(q_2)}_{\text{joint 2}} \oplus \underbrace{\mathscr{T}_x(a_2)}_{\text{link 2}} \oplus \underbrace{\mathscr{R}_z(q_3)}_{\text{joint 3}} \oplus \underbrace{\mathscr{T}_x(a_3)}_{\text{link 3}} \tag{9}$$

and use the now familiar functionality to represent and work with this arm.

$$\xi(q) = \begin{bmatrix} \cos q_{123} & -\sin q_{123} & a_1 \cos q_1 + a_2 \cos q_{12} + a_3 \cos q_{123} \\ \sin q_{123} & \cos q_{123} & a_1 \sin q_1 + a_2 \sin q_{12} + a_3 \sin q_{123} \\ 0 & 0 & 1 \end{bmatrix} \tag{10}$$

where:

$$q_{123} = q_1 + q_2 + q_3 \tag{11}$$

2-Dimensional (Planar) Robotic Arms

- We can easily add a ***third*** (revolute) joint

$$\xi_e(\boldsymbol{q}) = \underbrace{\mathscr{R}_z(q_1)}_{\text{joint 1}} \oplus \underbrace{\mathscr{T}_x(a_1)}_{\text{link 1}} \oplus \underbrace{\mathscr{R}_z(q_2)}_{\text{joint 2}} \oplus \underbrace{\mathscr{T}_x(a_2)}_{\text{link 2}} \oplus \underbrace{\mathscr{R}_z(q_3)}_{\text{joint 3}} \oplus \underbrace{\mathscr{T}_x(a_3)}_{\text{link 3}} \tag{9}$$

and use the now familiar functionality to represent and work with this arm.

$$\xi(q) = \begin{bmatrix} \cos q_{123} & -\sin q_{123} & a_1 \cos q_1 + a_2 \cos q_{12} + a_3 \cos q_{123} \\ \sin q_{123} & \cos q_{123} & a_1 \sin q_1 + a_2 \sin q_{12} + a_3 \sin q_{123} \\ 0 & 0 & 1 \end{bmatrix} \tag{10}$$

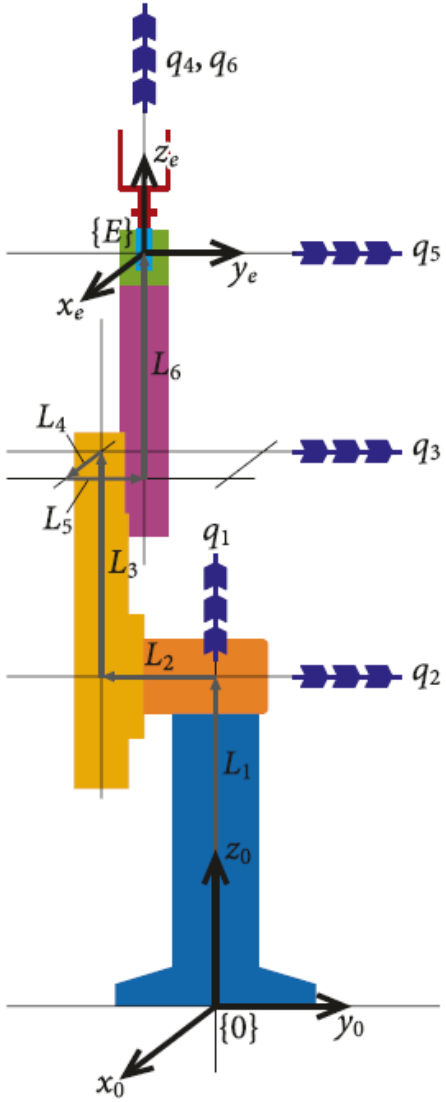
- This robot has 3 degrees of freedom and is able to access all points in the task space $\mathcal{T} \subset \mathbf{SE}(2)$, that is, achieve any pose in the plane (limited by reach).

3-Dimensional (Planar) Robotic Arms

- Truly useful *robot arms* have a **task space** $\mathcal{T} \subset \mathbf{SE}(3)$ enabling arbitrary position and orientation of the end-effector.
- This requires a robot arm with a ***configuration space*** $\dim \mathcal{C} \geq \dim \mathcal{T}$ which can be achieved by a robot with 6 (six) or *more* joints.

3-Dimensional (Planar) Robotic Arms

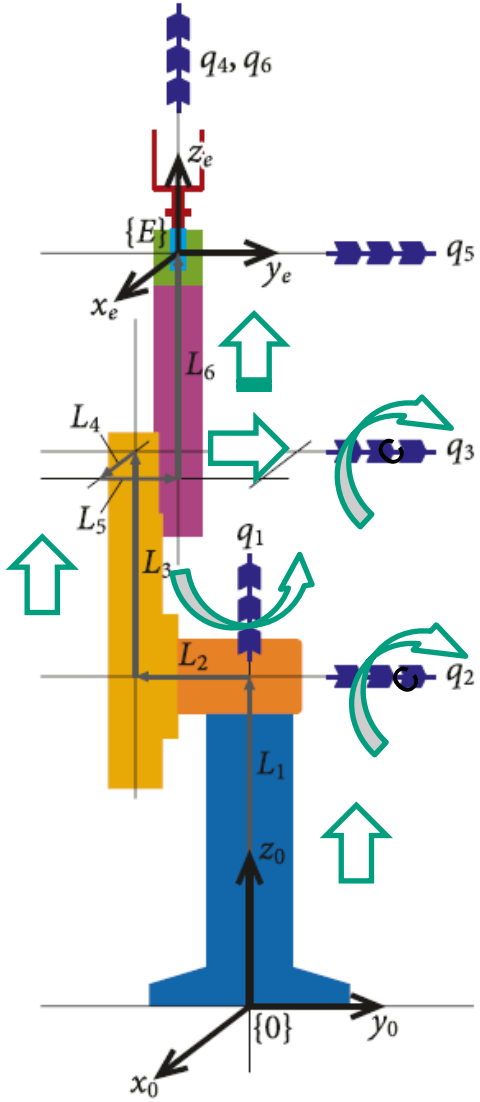
- We can ***extend*** the technique from the previous section for a robot like the Puma 560



Puma robot in the zero joint-angle configuration showing dimensions and joint axes.

3-Dimensional (Planar) Robotic Arms

- We can ***extend*** the technique from the previous section for a robot like the Puma 560
- Starting with the ***world frame*** $\{0\}$ we move up, rotate about the ***waist axis*** (q_1), rotate about the ***shoulder axis*** (q_2), move to the left, move up and so on.



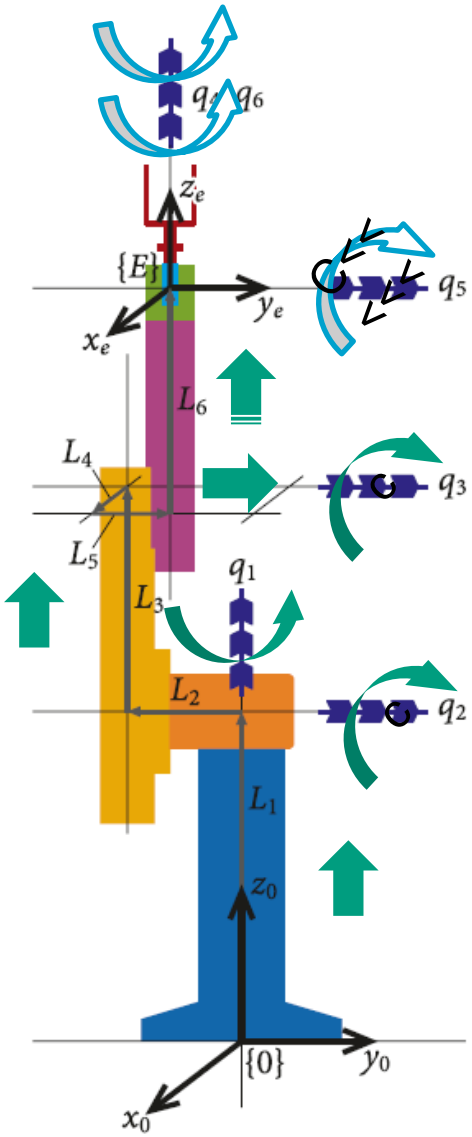
Puma robot in the zero joint-angle configuration showing dimensions and joint axes.

3-Dimensional (Planar) Robotic Arms

- We can ***extend*** the technique from the previous section for a robot like the Puma 560
- Starting with the ***world frame*** $\{0\}$ we move up, rotate about the ***waist axis*** (q_1), rotate about the ***shoulder axis*** (q_2), move to the left, move up and so on.

$$\xi_E = \mathscr{T}_z(L_1) \oplus \mathscr{R}_z(q_1) \oplus \mathscr{R}_y(q_2) \oplus \mathscr{T}_y(L_2) \oplus \mathscr{T}_z(L_3) \oplus \mathscr{R}_y(q_3)$$

$$\oplus \mathscr{T}_x(L_4) \oplus \mathscr{T}_y(L_5) \oplus \mathscr{T}_z(L_6) \oplus \underbrace{\mathscr{R}_z(q_4) \oplus \mathscr{R}_y(q_5) \oplus \mathscr{R}_z(q_6)}_{\text{wrist}}$$



Puma robot in the zero joint-angle configuration showing dimensions and joint axes.

3-Dimensional (Planar) Robotic Arms

- Then, we write down the complete transform expression:

$$\begin{aligned}\xi_E = {} & \mathscr{T}_z(L_1) \oplus \mathscr{R}_z(q_1) \oplus \mathscr{R}_y(q_2) \oplus \mathscr{T}_y(L_2) \oplus \mathscr{T}_z(L_3) \oplus \mathscr{R}_y(q_3) \\ & \oplus \mathscr{T}_x(L_4) \oplus \mathscr{T}_y(L_5) \oplus \mathscr{T}_z(L_6) \oplus \underbrace{\mathscr{R}_z(q_4) \oplus \mathscr{R}_y(q_5) \oplus \mathscr{R}_z(q_6)}_{\text{wrist}}\end{aligned} \tag{12}$$

- The wrist term represents a ZYZ Euler angle sequence and provides arbitrary orientation but is subject to a singularity when the middle angle $q_5 = 0$.

FK as a chain of Robot Links

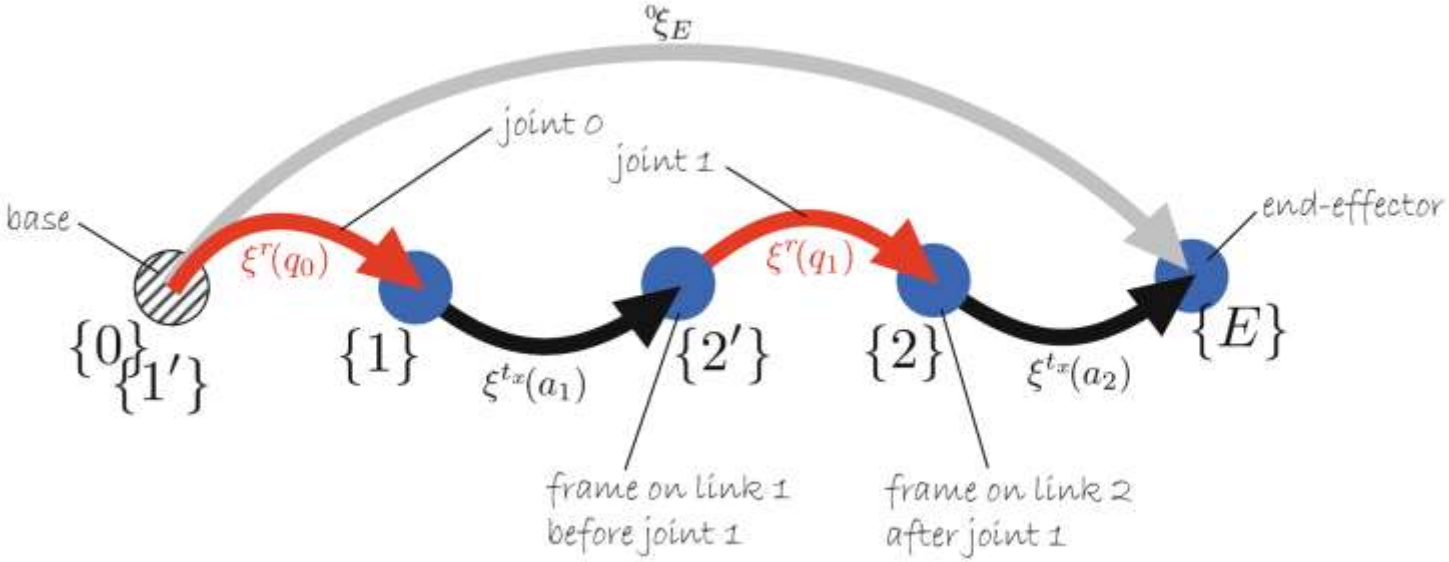
- The pose of the end effector is

$$ {}^{0}\xi_{N} = {}^{0}\xi_{1}(q_0) \oplus {}^{1}\xi_{2}(q_1) \oplus \cdots \oplus {}^{N-1}\xi_{N}(q_{N-1}) \tag{7.1} $$

where ${}^{j}\xi_{j+1}(q_j)$ is the pose of link frame {j+1} w.r.t. link {j}

- In functional form

$$ {}^{0}\xi_{N} = \mathcal{K}(\boldsymbol{q}) \tag{7.2} $$



3-Dimensional (Planar) Robotic Arms

- While this notation is somehow ***intuitive***, it does become ***cumbersome*** as the number of robot joints increases.

- Several approaches have been developed to more ***concisely*** describe the motion of a serial-link robotic arm:
  - Denavit-Hartenberg (DH) notation (Ch. 7.1.5)
  - Product of exponentials (not included in TEL200)
- While many other conventions have been developed, the DH convention remains the standard approach

Denavit-Hartenberg Parameters

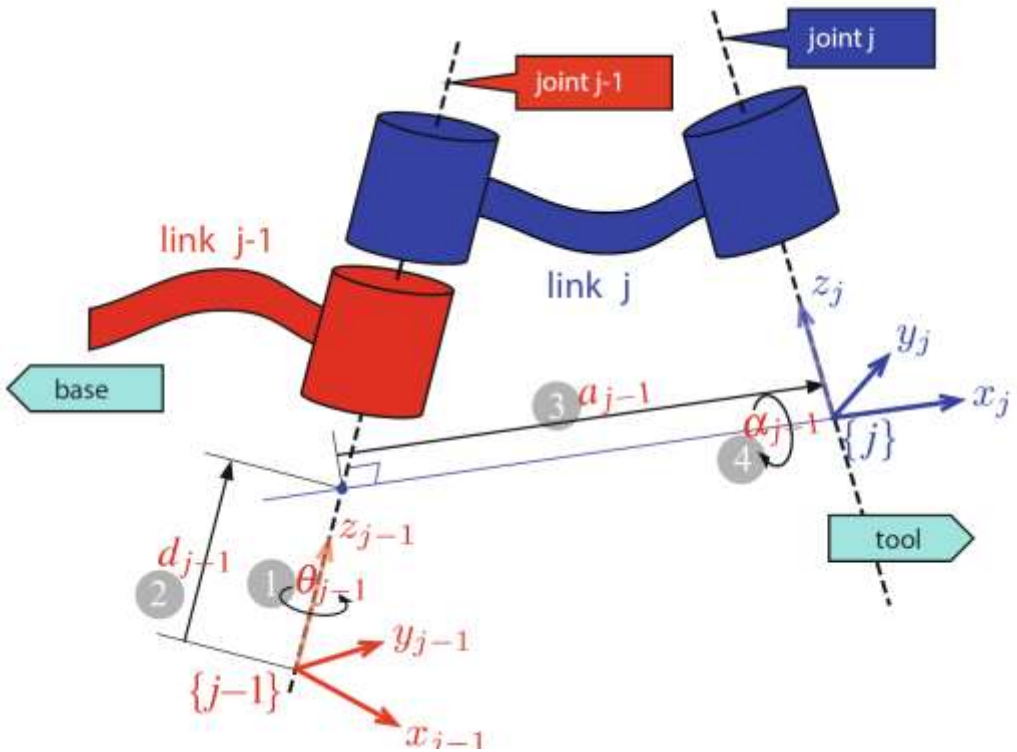
- One ***systematic*** way of describing the geometry of a serial chain of links and joints is Denavit-Hartenberg notation.

Joint angle	θ_j	The angle between the $\boldsymbol{x}_j$ and $\boldsymbol{x}_{j+1}$ axes about the $\boldsymbol{z}_{j-1}$ axis	Revolute joint variable or constant
Link offset	d_j	The distance from the origin of frame {j} to the $\boldsymbol{x}_{j+1}$ axis along the $\boldsymbol{z}_j$ axis	Constant or prismatic joint variable
Link length	a_j	The distance between the $\boldsymbol{z}_j$ and $\boldsymbol{z}_{j+1}$ axes along the $\boldsymbol{x}_{j+1}$ axis; for intersecting axes is parallel to $\boldsymbol{z}_j \times \boldsymbol{z}_{j+1}$	Constant
Link twist	α_j	The angle from the $\boldsymbol{z}_j$ axis to the $\boldsymbol{z}_{j+1}$ axis about the $\boldsymbol{x}_{j+1}$ axis	Constant
Joint type	σ_j	$\sigma = R$ for a revolute joint, $\sigma = P$ for a prismatic joint. By convention $R = 0$ and $P = 1$	Constant

Tab. 7.2 - Denavit-Hartenberg parameters: their symbol, physical meaning, and formal definition.

Denavit-Hartenberg Parameters

- One ***systematic*** way of describing the geometry of a serial chain of links and joints is Denavit-Hartenberg notation.



- The colors red and blue denote all things associated with links $j-1$ and j respectively;
- The numbers in circles 1 represent the order in which the elementary transforms are applied;
- The unit vector x_j is parallel to $z_{j-1} \times z_j$ and if those two axes are parallel then d_j can be chosen arbitrarily.

Fig. **7.18**: Definition of standard Denavit-Hartenberg link parameters.

Denavit-Hartenberg Parameters

- The transformation from link coordinate frame $\{j\}$ to frame $\{j+1\}$ is defined in terms of elementary ***rotations*** and ***translations*** as

$$ {}^{j}\boldsymbol{\xi}_{j+1} = \boldsymbol{\xi}^{r_z}(\theta_j) \oplus \boldsymbol{\xi}^{t_z}(d_j) \oplus \boldsymbol{\xi}^{t_x}(a_j) \oplus \boldsymbol{\xi}^{r_x}(\alpha_j) \tag{7.4} $$

which can be expanded in homogeneous matrix form in **SE**(3) as

$$ {}^{j}\mathbf{A}_{j+1} = \begin{pmatrix} \cos\theta_j & -\sin\theta_j\cos\alpha_j & \sin\theta_j\sin\alpha_j & a_j\cos\theta_j \\ \sin\theta_j & \cos\theta_j\cos\alpha_j & -\cos\theta_j\sin\alpha_j & a_j\sin\theta_j \\ 0 & \sin\alpha_j & \cos\alpha_j & d_j \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{7.5} $$

Denavit-Hartenberg Parameters

- The transformation from link coordinate frame $\{j\}$ to frame $\{j+1\}$ is defined in terms of elementary ***rotations*** and ***translations*** as

$$ {}^{j}\boldsymbol{\xi}_{j+1} = \boldsymbol{\xi}^{r_z}(\theta_j) \oplus \boldsymbol{\xi}^{t_z}(d_j) \oplus \boldsymbol{\xi}^{t_x}(a_j) \oplus \boldsymbol{\xi}^{r_x}(\alpha_j) \tag{7.4} $$

which can be expanded in homogeneous matrix form in **SE**(3) as

$$ {}^{j}\mathbf{A}_{j+1} = \begin{pmatrix} \cos\theta_j & -\sin\theta_j\cos\alpha_j & \sin\theta_j\sin\alpha_j & a_j\cos\theta_j \\ \sin\theta_j & \cos\theta_j\cos\alpha_j & -\cos\theta_j\sin\alpha_j & a_j\sin\theta_j \\ 0 & \sin\alpha_j & \cos\alpha_j & d_j \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{7.5} $$

Parameters α_j and a_j are always ***constant***

Denavit-Hartenberg Parameters

- The transformation from link coordinate frame $\{j\}$ to frame $\{j+1\}$ is defined in terms of elementary ***rotations*** and ***translations*** as

$$ {}^{j}\boldsymbol{\xi}_{j+1} = \boldsymbol{\xi}^{r_z}(\theta_j) \oplus \boldsymbol{\xi}^{t_z}(d_j) \oplus \boldsymbol{\xi}^{t_x}(a_j) \oplus \boldsymbol{\xi}^{r_x}(\alpha_j) \tag{7.4} $$

which can be expanded in homogeneous matrix form in **SE**(3) as

$$ {}^{j}\mathbf{A}_{j+1} = \begin{pmatrix} \cos\theta_j & -\sin\theta_j\cos\alpha_j & \sin\theta_j\sin\alpha_j & a_j\cos\theta_j \\ \sin\theta_j & \cos\theta_j\cos\alpha_j & -\cos\theta_j\sin\alpha_j & a_j\sin\theta_j \\ 0 & \sin\alpha_j & \cos\alpha_j & d_j \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{7.5} $$

For a ***revolute joint***, θ_j is the joint variable and d_j is ***constant***

Denavit-Hartenberg Parameters

- The transformation from link coordinate frame $\{j\}$ to frame $\{j+1\}$ is defined in terms of elementary ***rotations*** and ***translations*** as

$$ {}^{j}\boldsymbol{\xi}_{j+1} = \boldsymbol{\xi}^{r_z}(\theta_j) \oplus \boldsymbol{\xi}^{t_z}(d_j) \oplus \boldsymbol{\xi}^{t_x}(a_j) \oplus \boldsymbol{\xi}^{r_x}(\alpha_j) \tag{7.4} $$

which can be expanded in homogeneous matrix form in **SE**(3) as

$$ {}^{j}\mathbf{A}_{j+1} = \begin{pmatrix} \cos\theta_j & -\sin\theta_j\cos\alpha_j & \sin\theta_j\sin\alpha_j & a_j\cos\theta_j \\ \sin\theta_j & \cos\theta_j\cos\alpha_j & -\cos\theta_j\sin\alpha_j & a_j\sin\theta_j \\ 0 & \sin\alpha_j & \cos\alpha_j & d_j \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{7.5} $$

For a ***prismatic joint***, d_j is the joint variable, θ_j is ***constant*** *and* $\alpha_j = 0$

Denavit-Hartenberg Parameters

- The transformation from link coordinate frame $\{j\}$ to frame $\{j+1\}$ is defined in terms of elementary ***rotations*** and ***translations*** as

$$ {}^{j}\boldsymbol{\xi}_{j+1} = \boldsymbol{\xi}^{r_z}(\theta_j) \oplus \boldsymbol{\xi}^{t_z}(d_j) \oplus \boldsymbol{\xi}^{t_x}(a_j) \oplus \boldsymbol{\xi}^{r_x}(\alpha_j) \tag{7.4} $$

which can be expanded in homogeneous matrix form in **SE**(3) as

$$ {}^{j}\mathbf{A}_{j+1} = \begin{pmatrix} \cos\theta_j & -\sin\theta_j\cos\alpha_j & \sin\theta_j\sin\alpha_j & a_j\cos\theta_j \\ \sin\theta_j & \cos\theta_j\cos\alpha_j & -\cos\theta_j\sin\alpha_j & a_j\sin\theta_j \\ 0 & \sin\alpha_j & \cos\alpha_j & d_j \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{7.5} $$

$$ \boxed{q_j = \begin{cases} \theta_j & \text{if } \sigma_j = R \\ d_j & \text{if } \sigma_j = P \end{cases}} $$

Denavit-Hartenberg Parameters

- Using DH convention, a link is specified by only four parameters while it in general would entail six parameters; three each for translation and rotation.
- This is due to the two imposed constraints:

1. Position: axis x_j intersects z_{j-1}
2. Orientation: axis x_j is perpendicular to z_{j-1}.

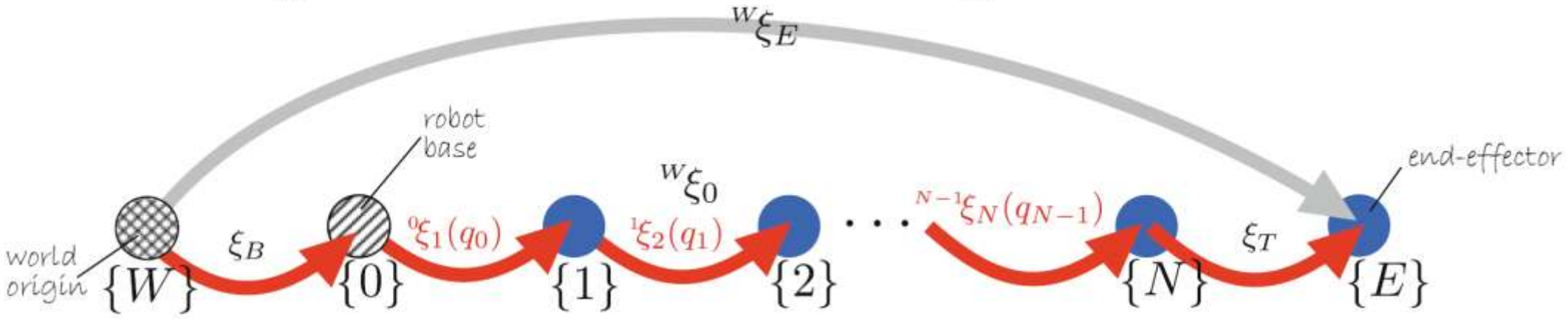
Consequently:

1. The link coordinate frame might be outside the actual, physical robot links!
2. The robot must have a particular “zero-angle configuration” that might even be mechanically unachievable!

FK as a chain of Robot Links

- Useful to add tools and base by extending FK expression of (7.1) with 2 transforms:

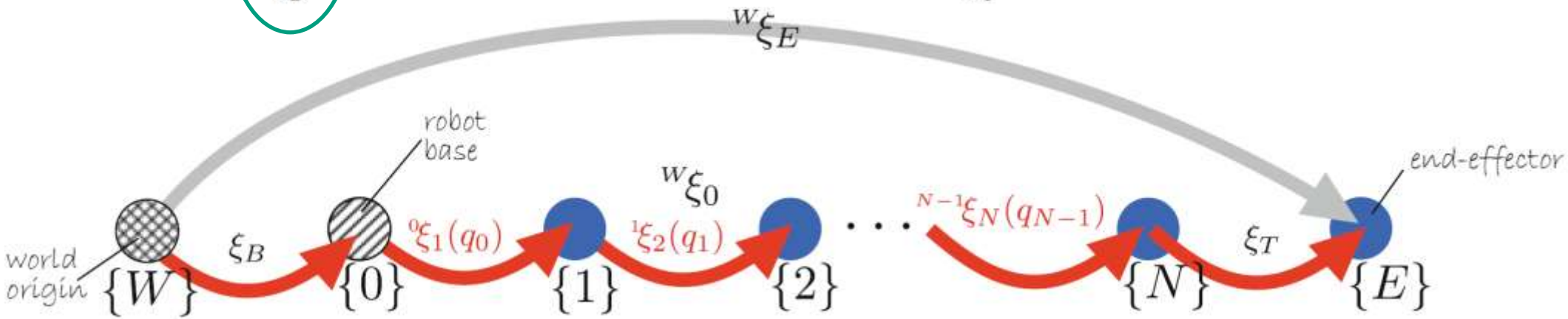
$$ {}^{W}\xi_{E} = \underbrace{{}^{W}\xi_{0}}_{\xi_B} \oplus {}^{0}\xi_{1} \oplus {}^{1}\xi_{2} \oplus \cdots \oplus {}^{N-1}\xi_{N} \oplus \underbrace{{}^{N}\xi_{E}}_{\xi_T} \qquad (7.3) $$



FK as a chain of Robot Links

- Useful to add tools and base by extending FK expression of (7.1) with 2 transforms:

$$ {}^{W}\xi_{E} = \underbrace{{}^{W}\xi_{0}}_{\xi_B} \oplus {}^{0}\xi_{1} \oplus {}^{1}\xi_{2} \oplus \cdots \oplus {}^{N-1}\xi_{N} \oplus \underbrace{{}^{N}\xi_{E}}_{\xi_T} \qquad (7.3) $$

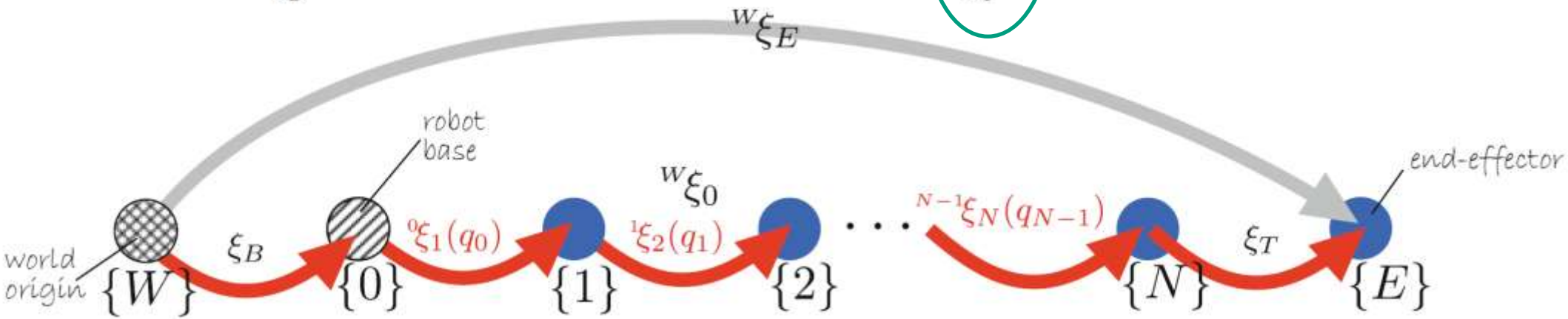


The base transform ξ_B puts the base of the robot arm at an arbitrary pose within the world coordinate frame.

FK as a chain of Robot Links

- Useful to add tools and base by extending FK expression of (7.1) with 2 transforms:

$$ {}^{W}\xi_{E} = \underbrace{{}^{W}\xi_{0}}_{\xi_B} \oplus {}^{0}\xi_{1} \oplus {}^{1}\xi_{2} \oplus \cdots \oplus {}^{N-1}\xi_{N} \oplus \underbrace{{}^{N}\xi_{E}}_{\xi_T} \qquad (7.3) $$



The frame $\{N\}$ is often defined as the ***center*** of the spherical wrist mechanism, and the tool transform ξ_T describes the pose of the **tool tip** with respect to that.

Robotics Toolbox - Denavit-Hartenberg (DH)

- Determining the DH parameters for a particular robot is challenging, but the Robotics Toolbox includes many robot models

```
>>> models.list(type="DH")
```

class	name	manufacturer	type	DoF	dims	st
Panda	Panda	Franka Emika	DH	7	3d	RRR
Puma560	Puma 560	Unimation	DH	6	3d	RRR
Stanford	Stanford arm	Victor Scheinman	DH	6	3d	RRP
Ball	ball		DH	10	3d	RRR
Hyper	Hyper10		DH	10	3d	RRR
Coil	Coil10		DH	10	3d	RRR
Cobra600	Cobra600	Omron	DH	4	3d	RRP
IRB140	IRB 140	ABB	DH	6	3d	RRR
KR5	KR5	KUKA	DH	6	3d	RRR
Orion5	Orion 5	RAWR Robotics	DH	4	3d	RRR
Planar3	Planar 3 link		DH	3	3d	RRR
Planar2	Planar 2 link		DH	2	3d	RR
LWR4	LWR-IV	Kuka	DH	7	3d	RRR
Sawyer	Sawyer	Rethink Robotics	DH	6	3d	RRR

*The result of this method is a wide table which has been cropped for inclusion in this book.

Robotics Toolbox - Denavit-Hartenberg (DH)

>>> irb140 = models.DH.IRB140();

```
>>> irb140
DHRobot: IRB 140 (by ABB), 6 joints (RRRRRR), dynamics, geometry, ...
```

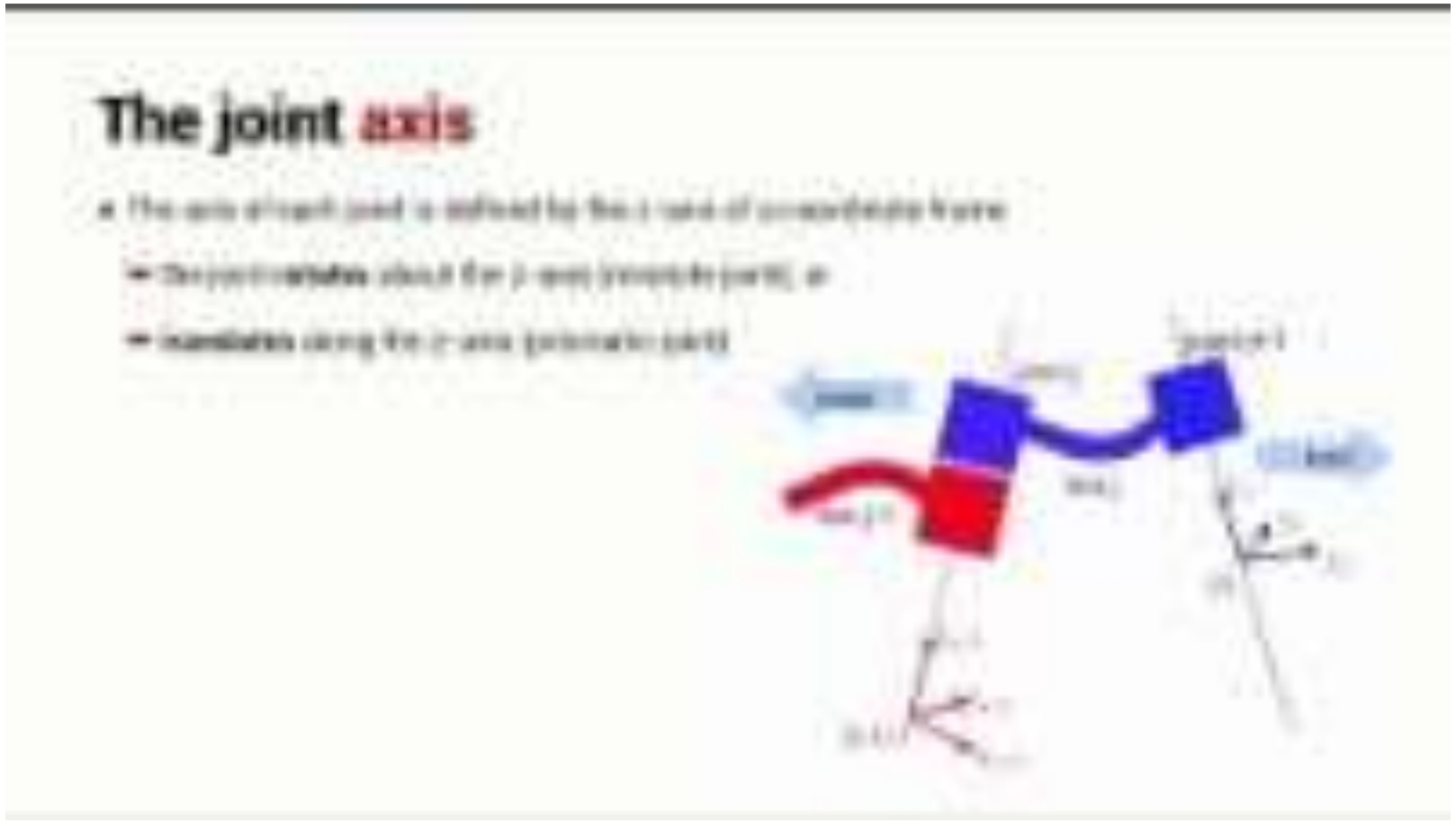
θ_j	d_j	a_j	α_j	q^-	q^+
q1	0.352	0.07	-90.0°	-180.0°	180.0°
q2	0	0.36	0.0°	-100.0°	100.0°
q3	0	0	-90.0°	-220.0°	60.0°
q4	0.38	0	90.0°	-200.0°	200.0°
q5	0	0	-90.0°	-120.0°	120.0°
q6	0.065	0	0.0°	-400.0°	400.0°

name	q0	q1	q2	q3	q4	q5	
qr	0°	-90°	90°	0°	90°	-90°	ready
qz	0°	0°	0°	0°	0°	0°	zero
qd	0°	-90°	180°	0°	0°	-90°	dextrous/normal

Denavit-Hartenberg convention – Ethan Tira-Thompson

- Video clip:

Denavit-Hartenberg convention – Peter Corke



NB. RVC Ed 3 (Python) uses **Zero-Based Indexing**, i.e. indexed starting from zero (video uses 1-based, form Ed 2)

Inverse Kinematics (Ch. 7.2)

- We have shown how to determine the pose of the end-effector given the *joint coordinates* and optional tool and base transforms.

- Another problem with real practical interest is the ***inverse problem***:

> Given the desired pose of the end-effector ξ_E what are the required joint coordinates $\boldsymbol{q}$?

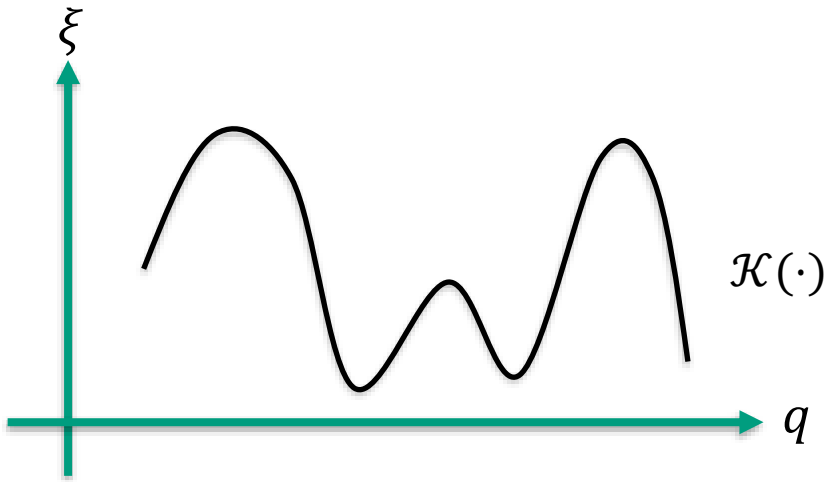
$$\begin{tikzcd} \mathbf{q} \arrow[r, bend left, "\text{Forward kinematics}"] & \xi_E \arrow[l, bend left, "\text{Backward kinematics}"] \end{tikzcd}$$

Inverse Kinematics

- This is called the **inverse kinematics problem** which is written in functional form as

$$\boldsymbol{q} = \mathcal{K}^{-1}(\boldsymbol{\xi}_E) \tag{7.6}$$

and, in general, this function is not unique, that is, several joint coordinate vectors $\boldsymbol{q}$ may result in the same end-effector pose ξ_E;

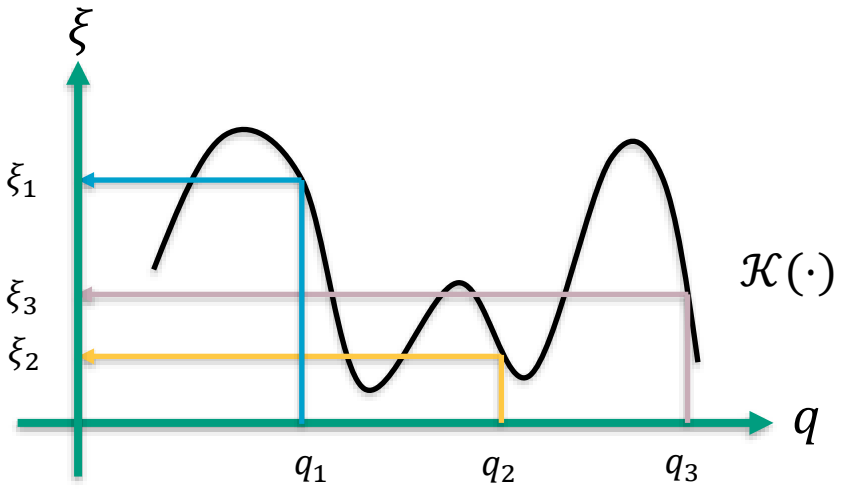


Inverse Kinematics

- This is called the **inverse kinematics problem** which is written in functional form as

$$\boldsymbol{q} = \mathcal{K}^{-1}(\boldsymbol{\xi}_E) \tag{7.6}$$

and, in general, this function is not unique, that is, several joint coordinate vectors $\boldsymbol{q}$ may result in the same end-effector pose ξ_E;



$\xi_1 = \mathcal{K}(q_1)$

$\xi_2 = \mathcal{K}(q_2)$

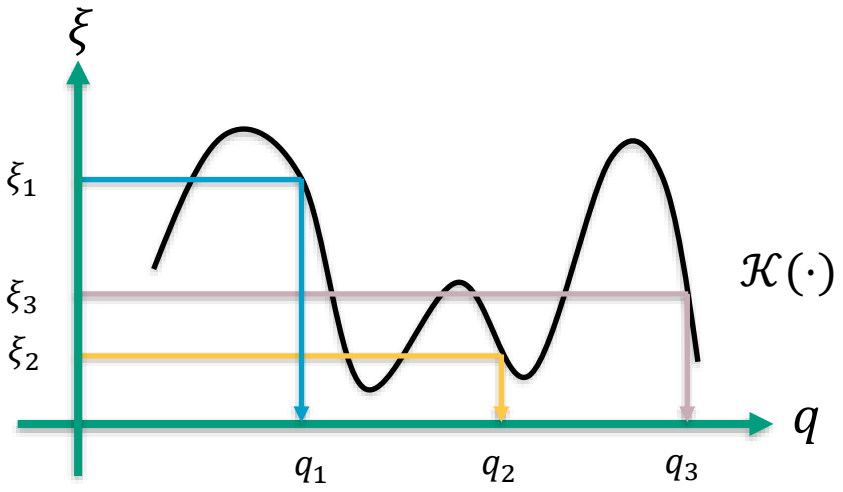
$\xi_3 = \mathcal{K}(q_3)$

Inverse Kinematics

- This is called the **inverse kinematics problem** which is written in functional form as

$$\boldsymbol{q} = \mathcal{K}^{-1}(\boldsymbol{\xi}_E) \tag{7.6}$$

and, in general, this function is not unique, that is, several joint coordinate vectors $\boldsymbol{q}$ may result in the same end-effector pose ξ_E;



$q_1 = \mathcal{K}^{-1}(\xi_1)$

$q_2 = \mathcal{K}^{-1}(\xi_2)$

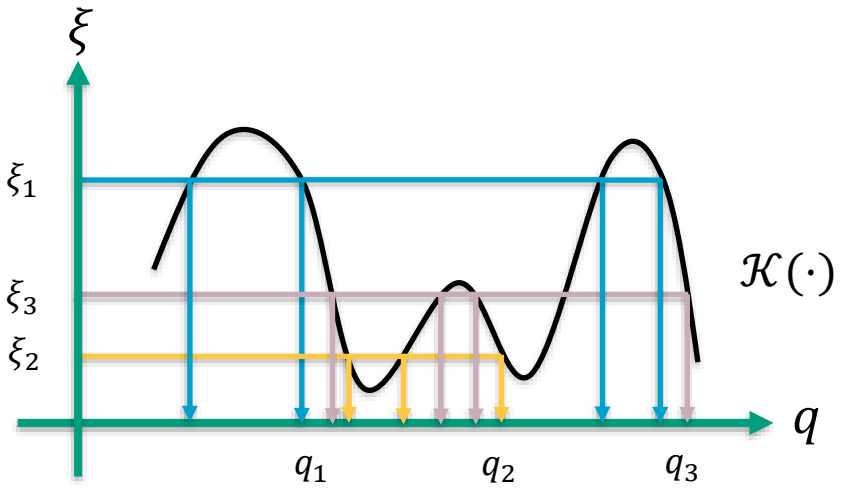
$q_3 = \mathcal{K}^{-1}(\xi_3)$

Inverse Kinematics

- This is called the **inverse kinematics problem** which is written in functional form as

$$\boldsymbol{q} = \mathcal{K}^{-1}(\boldsymbol{\xi}_E) \tag{7.6}$$

and, in general, this function is not unique, that is, several joint coordinate vectors $\boldsymbol{q}$ may result in the same end-effector pose ξ_E;



$q_1 = \mathcal{K}^{-1}(\xi_1)$

$q_2 = \mathcal{K}^{-1}(\xi_2)$

$q_3 = \mathcal{K}^{-1}(\xi_3)$

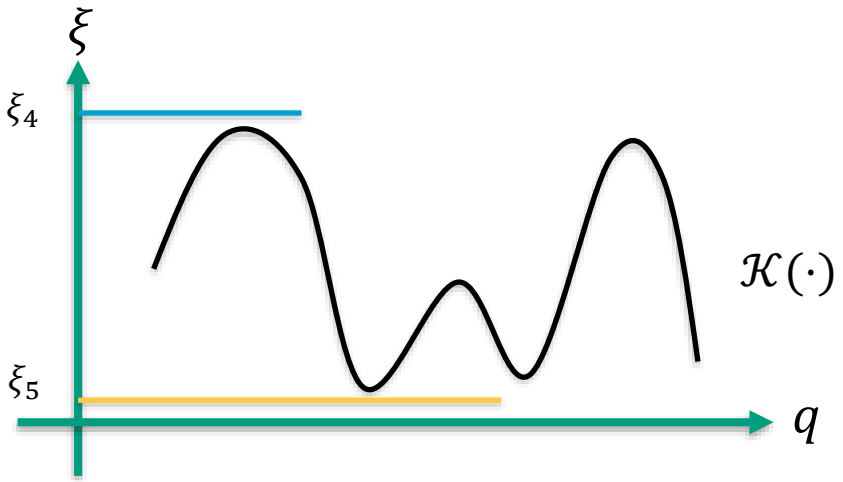
Multiple solutions !

Inverse Kinematics

- This is called the **inverse kinematics problem** which is written in functional form as

$$\boldsymbol{q} = \mathcal{K}^{-1}(\boldsymbol{\xi}_E) \tag{7.6}$$

and, in general, this function is not unique, that is, several joint coordinate vectors $\boldsymbol{q}$ may result in the same end-effector pose ξ_E;



$q_4 = \mathcal{K}^{-1}(\xi_4)$

$q_5 = \mathcal{K}^{-1}(\xi_5)$

No solutions !

Inverse Kinematics

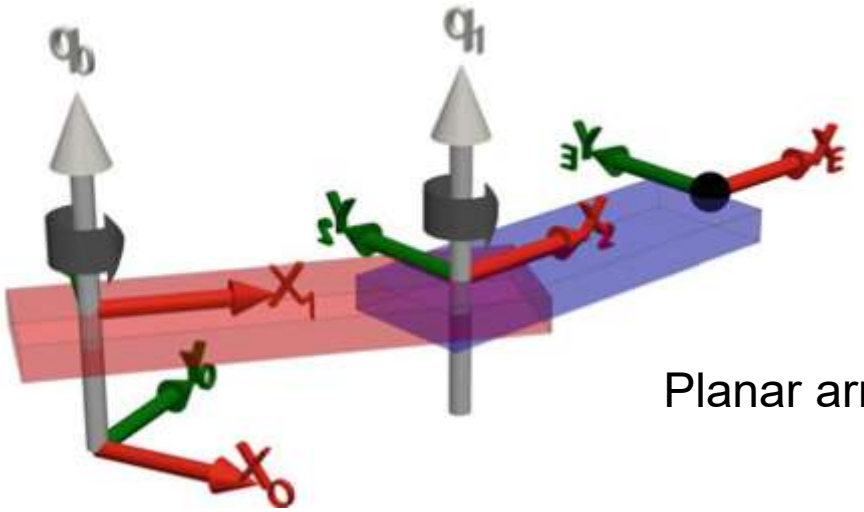
- Two main approaches can be used to determine the inverse kinematics.

  1. Closed form or analytical solution;

  2. An iterative numerical solution.

2-Dimensional (Planar) Robotic Arms

- We will illustrate inverse kinematics for a simple 2-joint robot two ways:
  1. Algebraic closed-form
  2. Numerical solution



Planar arm with two rotational joints.

Given the desired pose of the end-effector ξ_E what are the required joint coordinates $\boldsymbol{q}_0$ and $\boldsymbol{q}_1$?

Closed-form Solution

- Compute the ***forward kinematics*** symbolically using SymPy:

```
>>> import sympy
>>> a1, a2 = sympy.symbols("a1 a2")
>>> e = ET2.R() * ET2.tx(a1) * ET2.R() * ET2.tx(a2);
```

- Define symbolic variables for the joint angles

```
>>> q0, q1 = sympy.symbols("q0 q1")
```

- Compute forward kinematics as an **SE**(2) matrix

```
>>> TE = e.fkine([q0, q1])
cos(q0 + q1) -sin(q0 + q1) a1*cos(q0) + a2*cos(q0 + q1)
sin(q0 + q1) cos(q0 + q1) a1*sin(q0) + a2*sin(q0 + q1)
0 0 1
```

Closed-form Solution

- Consider just the end-effector position

```
>>> x_fk, y_fk = TE.t;
```

- Define desired end-effector position using two more symbolic variables

```
>>> x, y = sympy.symbols("x y")
```

- Two equations and two unknowns: $x_{\text{fk}} = x, \;\; y_{\text{fk}} = y$
- SymPy need some help to solve such trigonometric equations
- Square both equations and add them: $x_{\text{fk}}^2 + y_{\text{fk}}^2 = x^2 + y^2$
- Rewrite as: $x_{\text{fk}}^2 + y_{\text{fk}}^2 - x^2 - y^2 = 0$

Closed-form Solution – Solving for q_1

```
>>> eq1 = (x_fk**2 + y_fk**2 - x**2 - y**2).trigsimp()
a1**2 + 2*a1*a2*cos(q1) + a2**2 - x**2 - y**2
>>> q1_sol = sympy.solve(eq1, q1)
[-acos(-(a1**2 + a2**2 - x**2 - y**2)/(2*a1*a2)) + 2*pi,
acos((-a1**2 - a2**2 + x**2 + y**2)/(2*a1*a2))
```

- Eq1 has only one unknown, q1
- q1_sol is a **list**.
- Correspont to pos or neg q1
- IK problem does not have unique solution

Closed-form Solution – Solving for q_0

- For q0, we first expand the two equations:

```
>>> eq0 = tuple(map(sympy.expand_trig, [x_fk - x, y_fk - y]))

(a1*cos(q0) + a2*(-sin(q0)*sin(q1) + cos(q0)*cos(q1)) - x,

a1*sin(q0) + a2*(sin(q0)*cos(q1) + sin(q1)*cos(q0)) - y)
```

- solve them for sin(q0) and cos(q0)

```
>>> q0_sol = sympy.solve(eq0, [sympy.sin(q0), sympy.cos(q0)]);
```

- The ratio of these is tan(q0)

```
>>> sympy.atan2(q0_sol[sympy.sin(q0)], q0_sol[sympy.cos(q0)]).simplify()

atan2((a1*y - a2*x*sin(q1) + a2*y*cos(q1))/(a1**2 + 2*a1*a2*cos(q1)+ a2**2),

(a1*x + a2*x*cos(q1) + a2*y*sin(q1))/(a1**2 + 2*a1*a2*cos(q1)+ a2**2))
```

Closed-form Solution

- SymPy is comparatively weak at trigonometric expressions
- More powerful symbolic tools exist
- The ***complexity*** of algebraic solution increases dramatically with the ***number of joints*** and more sophisticated symbolic solution approaches need to be used.
- In Matlab, the ikine_sym method generates **symbolic inverse kinematics** solutions for a limited class of robot manipulators.

Numerical Solution

- We can think of the ***inverse kinematics problem*** as one of adjusting the joint coordinates q until the forward kinematics $\mathcal{K}$ ***matches*** the desired pose p;

- More formally this is an ***optimization problem*** – to minimize the error between the ***forward kinematic*** solution and the ***desired pose*** ξ^*;

$$\boldsymbol{q}^* = \arg\min_{\boldsymbol{q}} \left\| \mathcal{K}(\boldsymbol{q}) \ominus \xi^* \right\| \tag{23}$$

- For our simple *2-link robot arm* example the error function comprises only the error in the end-effector position, *not* its *orientation*:

$$E(q) = \left\| \left[\mathcal{K}(\boldsymbol{q})\right]_t - \left(x^* \; y^*\right)^T \right\| \tag{24}$$

Numerical Solution

- We can solve this problem using the SciPy multi-variable minimization function

```
>>> pstar = np.array([0.6, 0.7]); # desired position

>>> E = lambda q: np.linalg.norm(e.fkine(q).t - pstar); #define error

>>> sol = optimize.minimize(E, [0, 0]);

>>> sol.x

array([ -0.2295, 2.183])
```

where the ***first argument*** is the error function, that incorporates the desired end-effector position;

…and the ***second argument*** is the initial guess the joint coordinates.

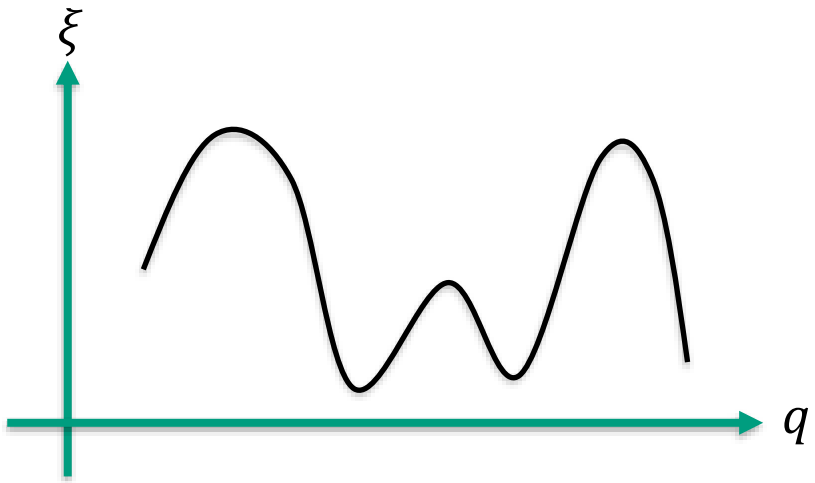
Numerical Solution

- The computed joint angles indeed give the ***desired*** end-effector position as:

```
>>> e.fkine(sol.x).printline()
```

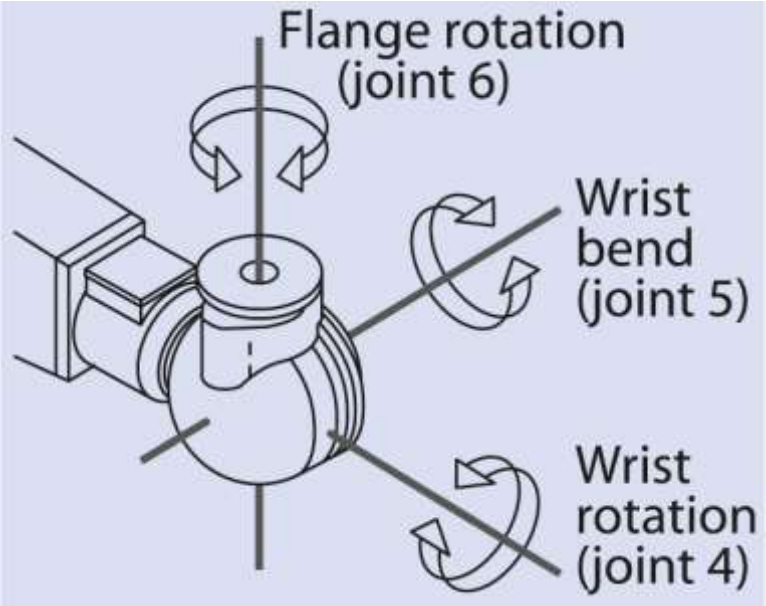
t = 0.6, 0.7; 112°

- As already discussed, there are *two* solutions for q but the solution that is found depends on the ***initial choice*** of q in `optimize.minimize`;



3-Dimensional Robotic Arms

- ***Closed-form*** solutions have been developed for most common types of 6-axis industrial robots and many are included in the Toolbox;
- A ***necessary condition*** for a closed-form solution of a 6-axis robot is a ***spherical wrist mechanism***.
- This is a gimbal-like mechanism: have a singularity.
- It doesn't cause any translation. So, position of end-effector is a function only of first three joints!
- An arbitrary end-effector orientation is achieved independently by means of the three wrist joints.



Inverser kinematics: analytical/closed-form solutions

- Can be solved in a fixed number of operations (therefore, computationally fast)
- Results in all possible solutions to the manipulator kinematics
- Most desirable for real-time control
- Often difficult or even impossible to find

Inverser kinematics: numerical solution

- Results in a numerical, iterative solution to system of equations (optimization problem solved e.g., using Newton/Raphson techniques)
- Unknown number of operations to solve
- Only returns a single solution
- Accuracy can be dictated by user but can prolong solution time
- Because of these reasons, this is much less desirable than a closed-form solution.
- Can be applied to all robots.

Inverse kinematics for 6-DOF robot

https://youtu.be/nLI6F6yjcfY

Forward and Inverse Kinematics

- Video clip:

https://youtu.be/SZP1KQ2qSEA

Trajectories (Ch. 7.3)

- One of the most ***common requirements*** in robotics is to move the robot end-effector smoothly from ***pose A*** to ***pose B;***

- Here, we will discuss *two* approaches to generating such trajectories: straight lines in ***configuration space*** and straight lines in ***task space***.

- These trajectories are known respectively as:
  - ❑ Cartesian motion (`MoveL` in RAPID)
  - ❑ Joint-space motion (`MoveJ/MoveAbsJ` in RAPID)

Joint-space Motion

- Consider the end-effector moving between *two* Cartesian poses.

```
>>> TE1 = SE3.Trans(0.4, -0.2, 0) * SE3.Rx(3);
>>> TE2 = SE3.Trans(0.4, 0.2, 0) * SE3.Rx(1);
```

which describe points in the xy-plane with ***different*** end-effector orientations.

Joint-space Motion

- The ***joint coordinate*** vectors associated with these poses are given by:

```
>>> sol1 = puma.ikine_a(TE1, "ru");

>>> sol2 = puma.ikine_a(TE2, "ru");
```

where we have specified the right-handed elbow-up (ru) configuration

- We require the motion to occur over a time period of ***2 seconds*** in 20 ms time steps:

```
>>> t = np.arange(0, 2, 0.02);
```

Joint-space Motion

- A ***joint-space trajectory*** is formed by smoothly interpolating between the joint configurations q_1 and q_2.

- The ***scalar interpolation*** functions quintic or trapezoidal can be used in conjunction with the multi-axis driver function mtraj as:

```
>>> traj = mtraj(quintic, sol1.q, sol2.q, t);
```

or

```
>>> traj = mtraj(trapezoidal, sol1.q, sol2.q, t);
```

which each result in a **trajectory *matrix*** $q \in \mathbb{R}^{100\times 6}$ with one row per ***time step*** and one column per ***joint***.

Joint-space Motion

- From here on we will use the equivalent jtraj convenience function

>>> traj = jtraj(sol1.q, sol2.q, t)

Trajectory created by jtraj: 100 time steps x 6 axes

- For mtraj and jtraj the final argument can be a time vector, as here, or an integer specifying the number of time steps.

- Function jtraj is equivalent to mtraj with quintic interpolation but optimized for the ***multi-axis case*** and also allowing initial- and final velocity to be set using additional arguments;

Joint-space Motion

- The trajectory is best viewed as an ***animation***

>>> puma.plot(traj.q);

but we can also plot the joint angles versus time

>>> xplot(t, traj.q);

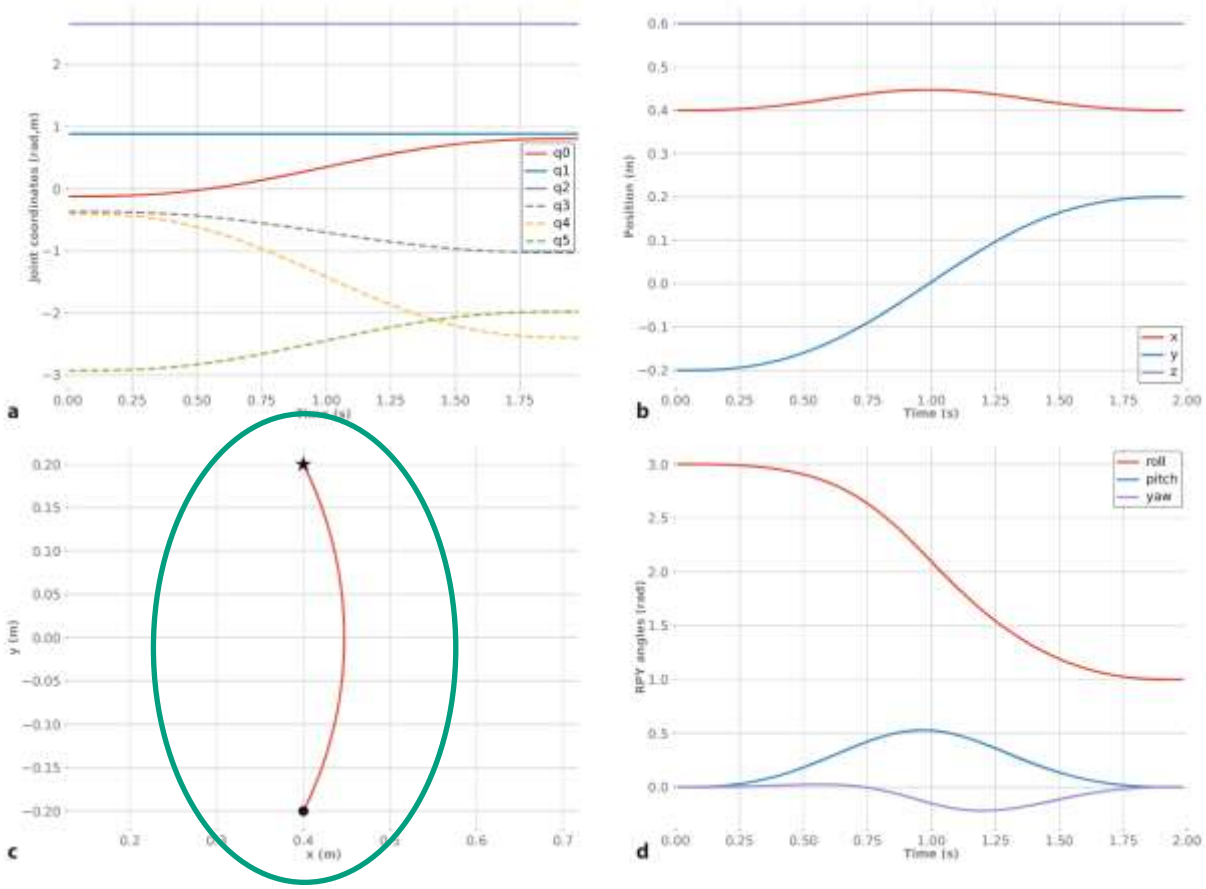


Fig. 7.20. Joint-space motion.
a Joint coordinates versus time;
b Cartesian position versus time;
c Cartesian position locus in the xy-plane
d roll-pitch-yaw angles versus time

Cartesian Motion

- For ***many applications*** we require straight-line motion in Cartesian space which is known as ***Cartesian motion***.
- This is implemented using the Toolbox function ctraj

```
>>> Ts = ctraj(TE1, TE2, t);
```

where the arguments are the initial and final pose, T1 and T2, and the *time steps* and it returns the trajectory as an array of SE3 objects.

Cartesian Motion

- As for the previous joint-space example, now we will extract and plot the ***translation***

```
>>> xplot(t, Ts.t, labels="x y z");
```

and ***orientation*** components

```
>>> xplot(t, Ts.rpy("xyz"), labels="roll pitch yaw");
```

of this motion which is shown in Fig. 7.21 along with the path of the end-effector in the xy-plane.

Cartesian Motion

- The corresponding ***joint-space trajectory*** is obtained by applying the inverse kinematics

>>> qc = puma.ikine_a(Ts);

and is shown in Fig. 7.21a.

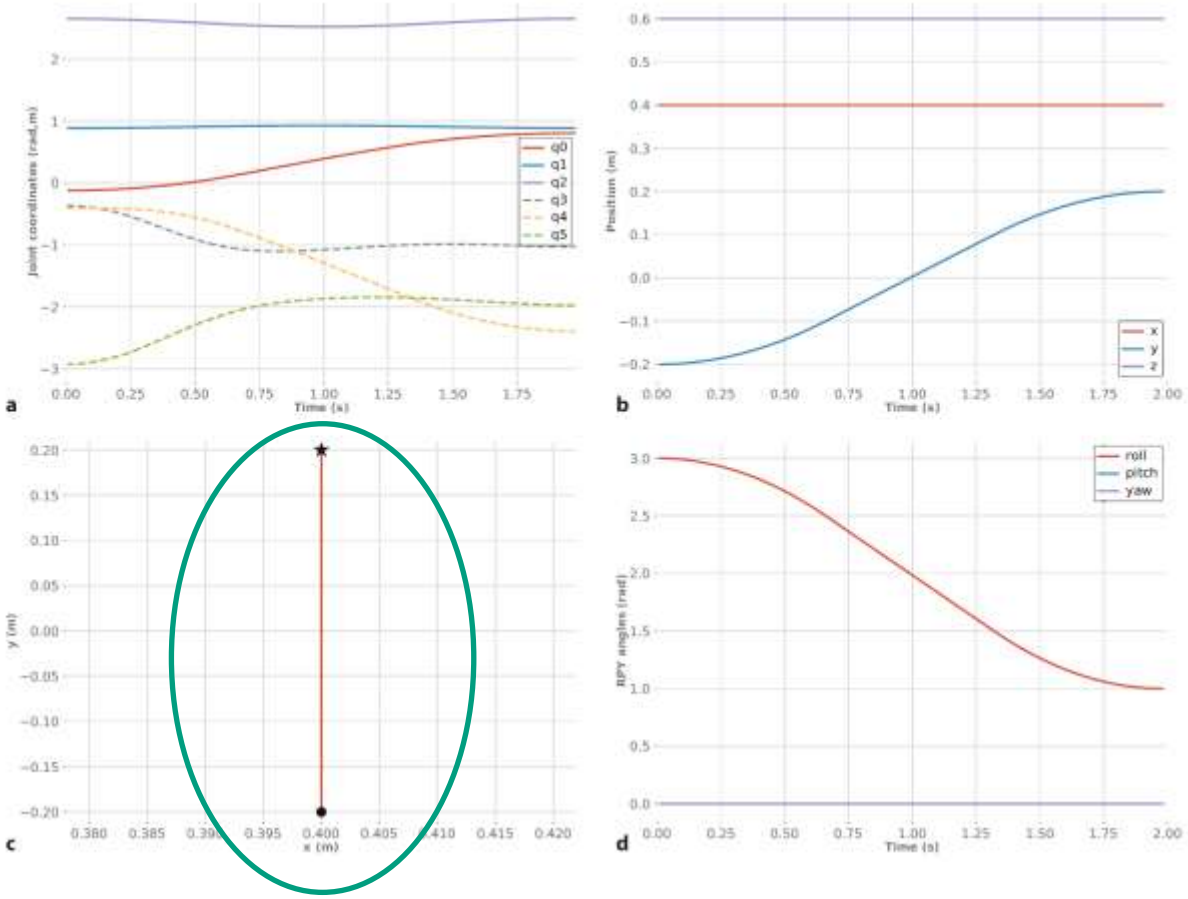


Fig. 7.21. Cartesian motion.
a Joint coordinates versus time;
b Cartesian position versus time;
c Cartesian position locus in the *xy*-plane;
d roll-pitch-yaw angles versus time

Motion through a Singularity

- Here, we deliberately choose a joint trajectory that moves through a robot ***wrist singularity***.
- We change the ***Cartesian endpoints*** of the previous example to

```
>>> TE1 = SE3.Trans(0.5, -0.3, 1.12) * SE3.OA((0, 1, 0), (1, 0, 0));

>>> TE2 = SE3.Trans(0.5, 0.3, 1.12) * SE3.OA((0, 1, 0), (1, 0, 0));
```

which results in motion in the y-direction with the end-effector z-axis pointing in the world x-direction.

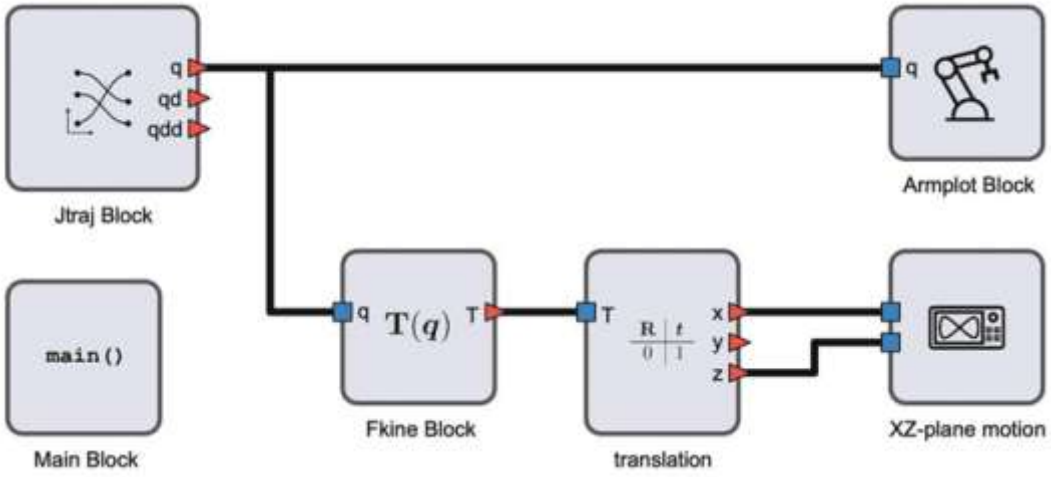


Fig. 7.22: Block diagram model models/jointspace for joint-space motion

Motion through a Singularity

- The ***Cartesian path*** is

```
>>> Ts = ctraj(TE1, TE2, t);
```

which we convert to ***joint coordinates***

```
>>> sol = puma.ikine_a(Ts, "lu");
```

And plot againt time in Fig. 7.23a.

```
>>> xplot(t, sol.q, unwrap=True);
```

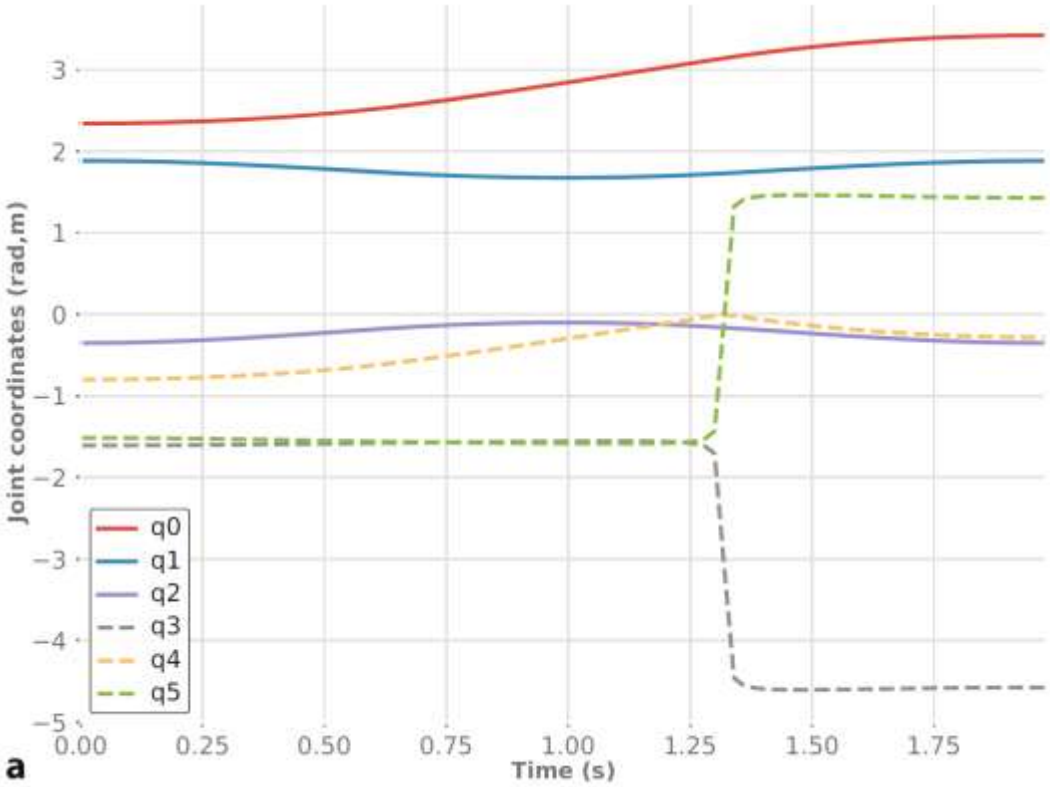


Fig. 7.23: Joint agnles through a ***wrist singularity***.
a Joint coordinates computed by the inverse kinematics function (`ikine6s`);

Motion through a Singularity

- At **time t1.3 s** wrist joint angles q3 and q5 change very rapidly.
- q5 increase, while q3 decrease rapidly
- **Counterrotational motion:** the gripper does not rotate but; **motors are working very hard**.

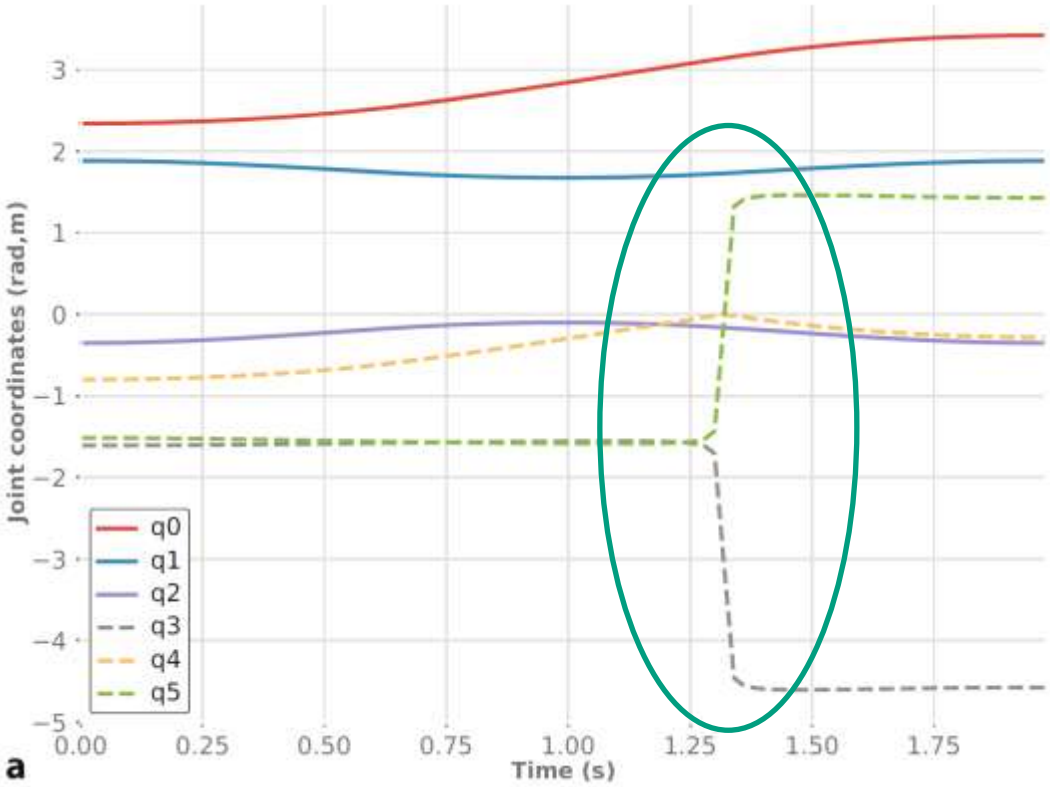


Fig. 7.23: Joint agnles through a ***wrist singularity***.
a Joint coordinates computed by the inverse kinematics function (`ikine6s`);

Motion through a Singularity

- This is since q4 is close to zero which means that the q3 and q5 rotational axes of the wrist are almost aligned…

…that is another ***gimbal lock*** situation or singularity.

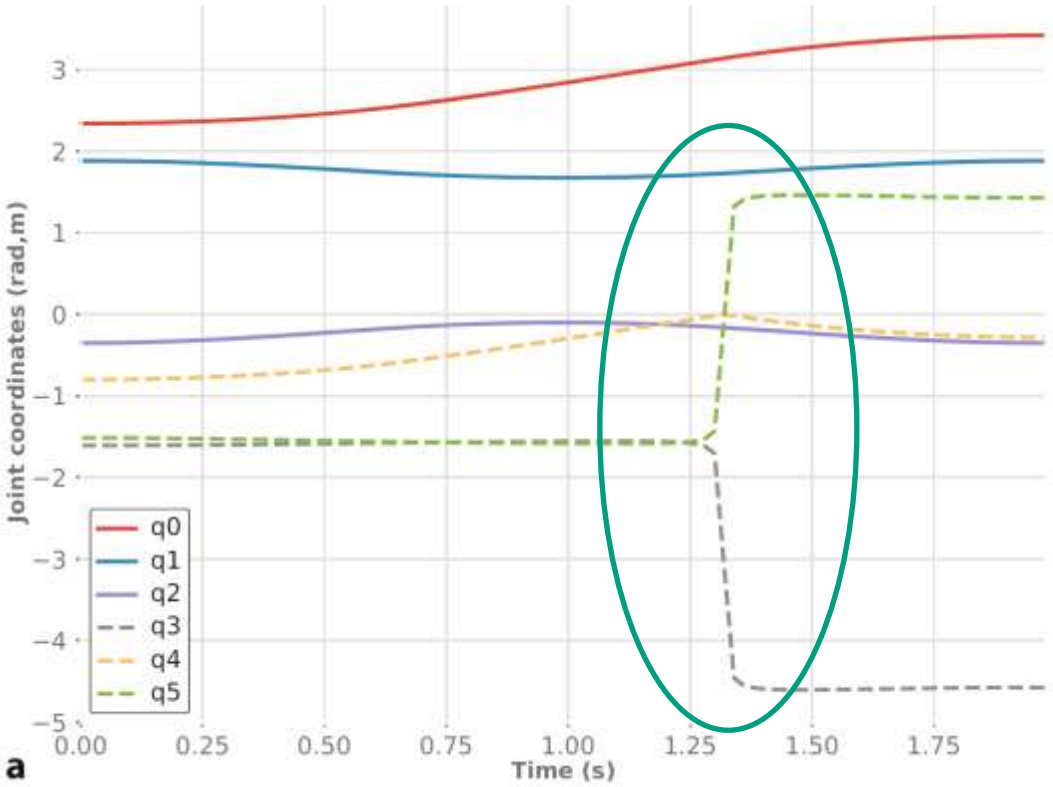


Fig. 7.23: Joint agnles through a ***wrist singularity***.
a Joint coordinates computed by the inverse kinematics function (`ikine6s`);

Motion through a Singularity

- The ***joint axis alignment*** means that the robot has lost 1 ***degree of freedom*** and is now effectively a 5-axis robot;
- Kinematically we can ***only*** solve for the sum $(q_3 + q_5)$ and there are an ***infinite number*** of solutions for q_3 and q_5 that would have the ***same*** sum.

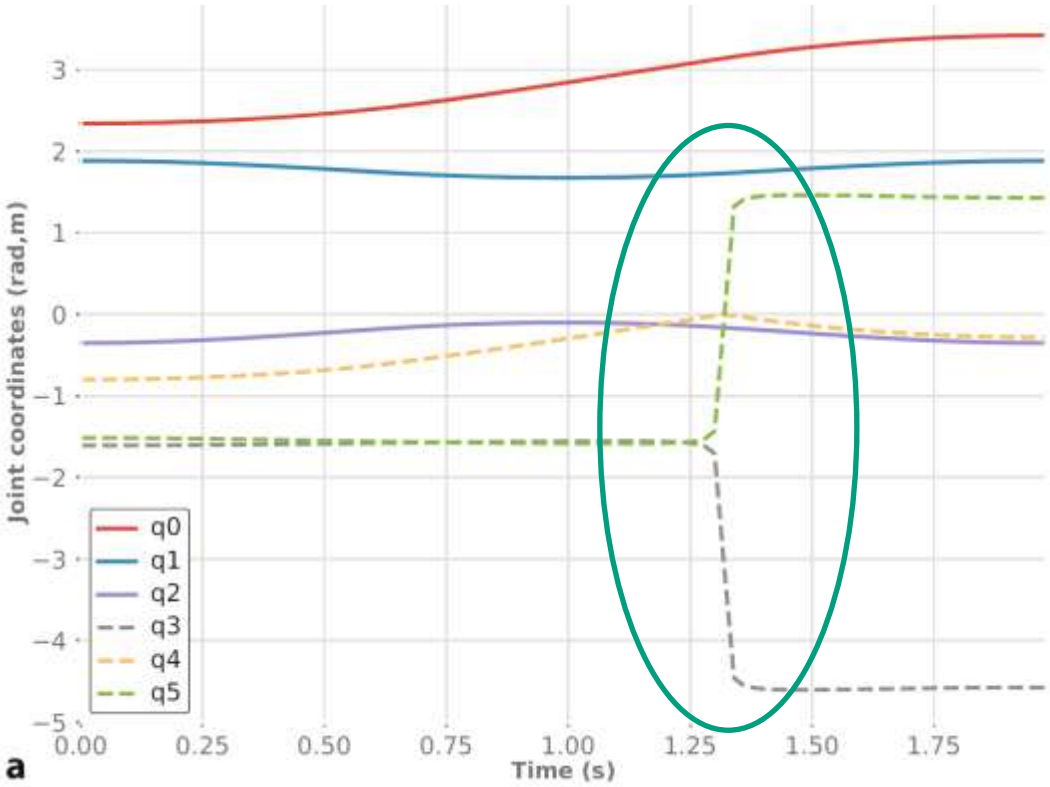


Fig. 7.23: Joint angles through a ***wrist singularity***.
a Joint coordinates computed by the inverse kinematics function (`ikine6s`);

Motion through a Singularity

- From Fig. 7.23b we observe that the ***numerical inverse kinematics*** method `ikine_LM` handles the ***singularity*** with ***far less*** unnecessary joint motion.

- This is a consequence of the ***minimum-norm solution*** which has returned the smallest magnitude q_3 and q_5 which have the correct sum.

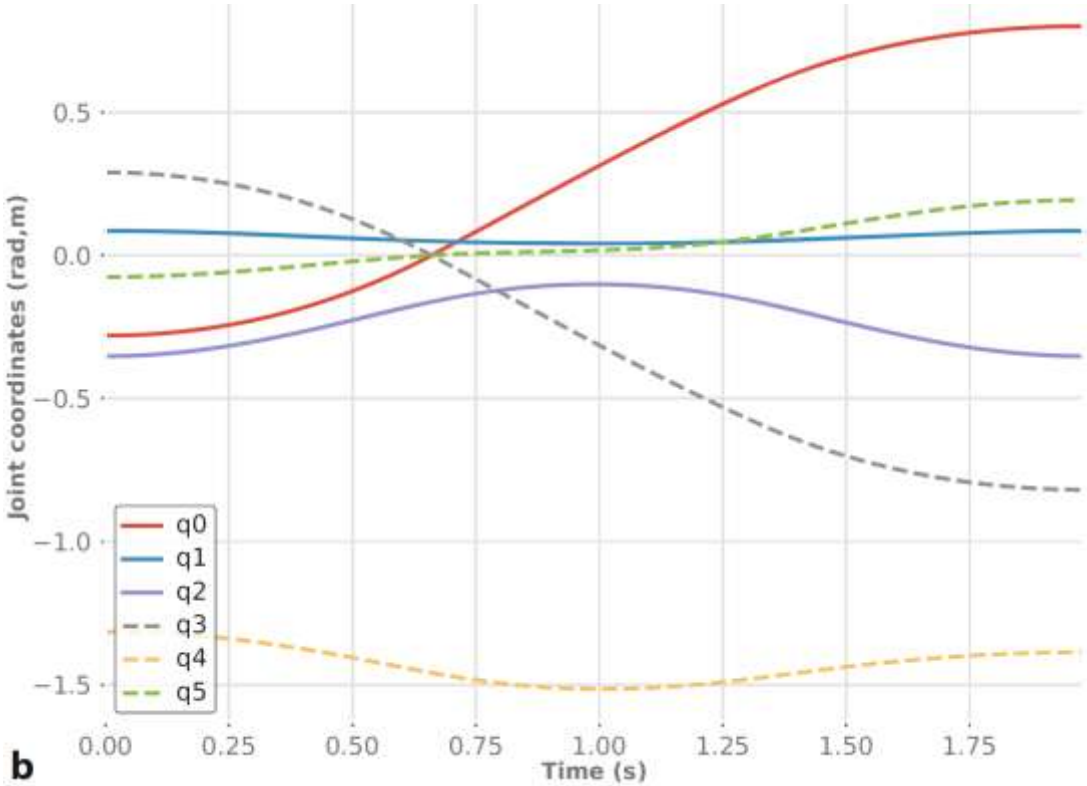


Fig. 7.23: Joint angles through a ***wrist singularity***.
b cartesian trajectory by the numerical inverse kinematics function (`ikine_LM`);

Motion through a Singularity

- The ***joint-space motion*** between the two poses, Fig. 7.23c, is immune to the singularity problem since it is does ***NOT*** involve inverse kinematics.
- However, it will not ***maintain*** the orientation of the tool in the x-direction for the whole path…

…only at the two ***end points***.

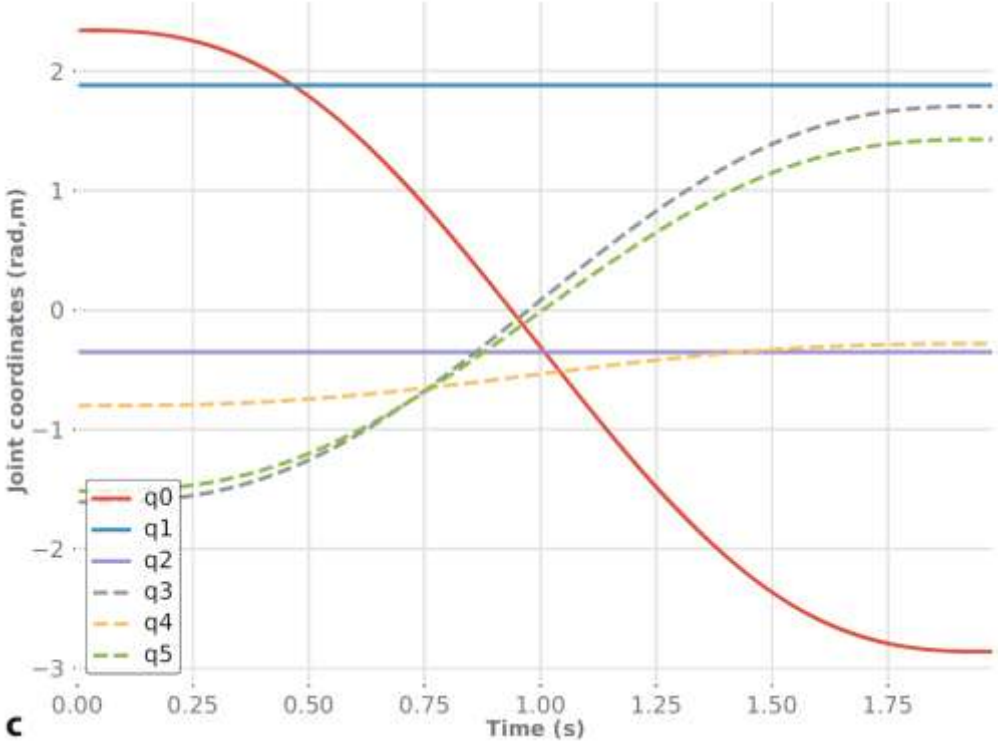


Fig. 7.23: Joint angles through a ***wrist singularity***.
c joint-space motion (`jtraj`);

Motion through a Singularity

- The ***dexterity*** of a robot manipulator, its ability to move easily in any direction, is referred to as its ***manipulability***.
- It is a ***scalar measure***, high is good, and can be computed for each point along the trajectory

```
>>> m = puma.manipulability(sol.q);
```

and is plotted in Fig. 7.23d.

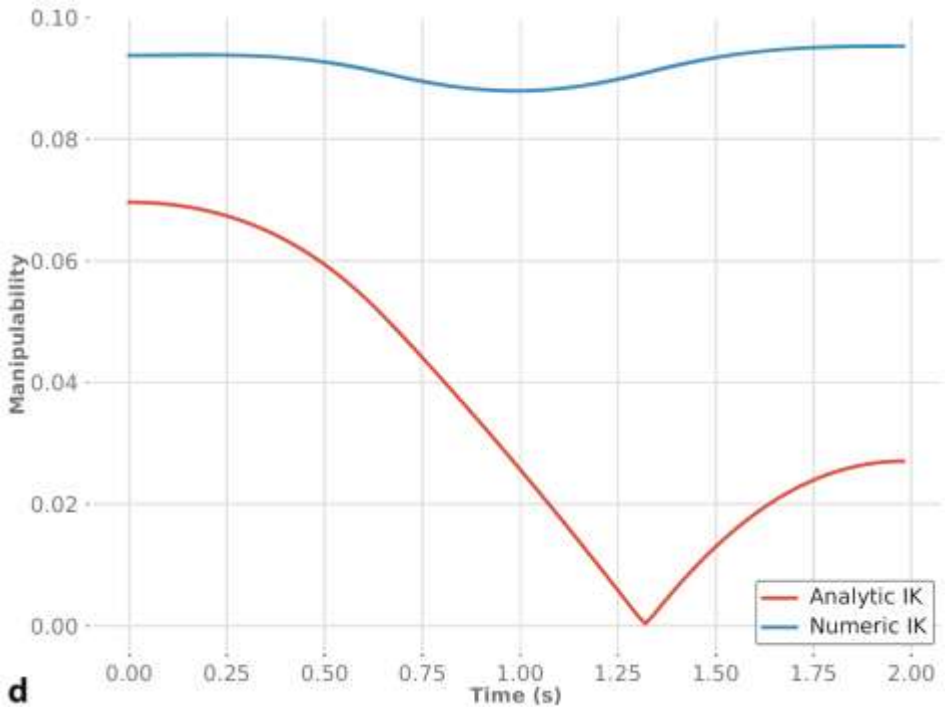


Fig. 7.23: Cartesian motion through a ***wrist singularity***. **d** manipulability;

Motion through a Singularity

- This shows that ***manipulability*** for analytic IK (red) was almost zero around the time of the rapid wrist joint motion.
- The numerical inverse kinematics (blue) was able to keep manipulability high throughout the trajectory since it implicitly minimizes the joint velocities.
- The ***manipulability measure*** *(w)* and the **numerical *inverse kinematics*** function are based on the manipulator's Jacobian matrix (J) which is the topic of the next chapter (Ch. 8).

$$w = \sqrt{\det(J(q)J^T(q))}$$

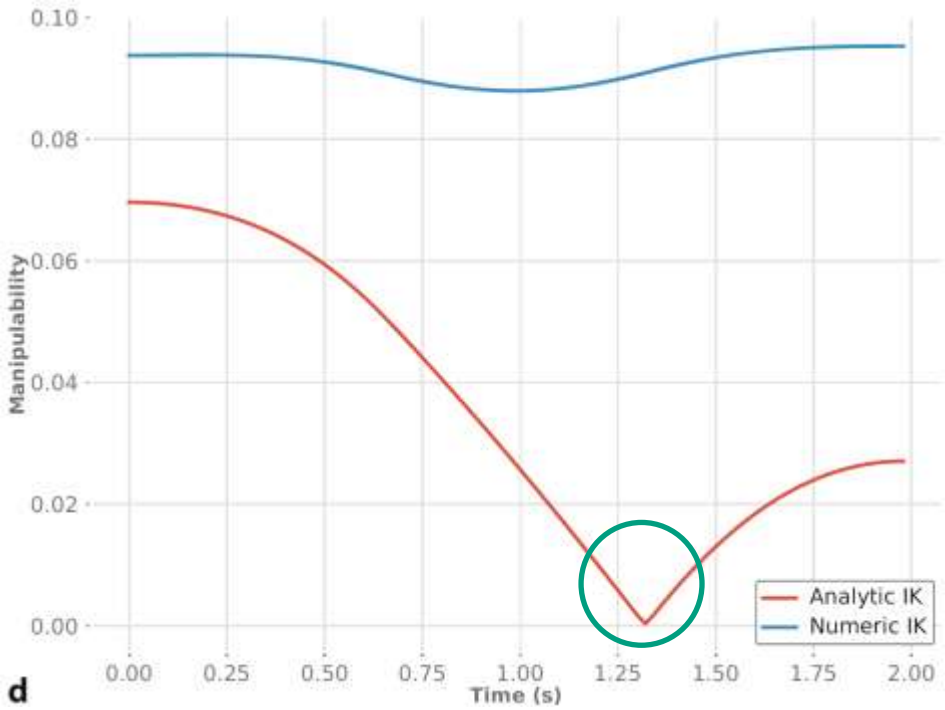


Fig. 7.23: Cartesian motion through a ***wrist singularity***. **d** manipulability;

Some further aspects:

- Configuration Change
  - ❑ Move from/to different configurations (*left-* or *right-handed,* elbow up or down, wrist flipped/non-flipped)
- Under-Actuated Manipulator
  - ❑ An ***under-actuated manipulator*** is one that has ***fewer*** than 6 (six) joints and is, therefore, limited in the end-effector poses that it can *attain*.
- Redudant Manipulator
  - ❑ A ***redundant manipulator*** is a robot with ***more than*** 6 (six) joints.

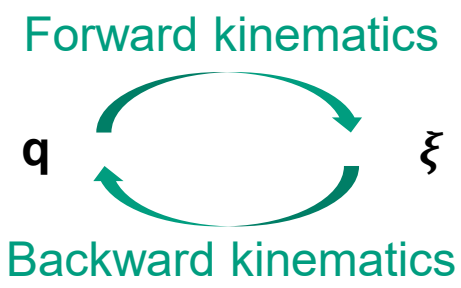
Wrapping up…

- We have learned *how* to determine the forward kinematics and the inverse kinematics of a *serial-link robot manipulator*;

- ***Forward kinematics*** involves determining the end-effector pose ξ given the joint coordinates $\boldsymbol{q}$;

- ***Inverse kinematics*** is the problem of determining the joint coordinates $\boldsymbol{q}$ given the end-effector pose $\boldsymbol{\xi}$;

- The ***joint*** and ***link structure*** can be expressed ***systematically*** by using four parameters for each link through the Denavit-Hartenberg (DH) convention;



Wrapping up…

- For ***simple robots***, or those with 6 (six) joints and a ***spherical wrist*** we can compute the ***inverse kinematics*** using an analytic solution;

…This inverse is not unique, and the robot may have ***several*** joint configurations that result in the ***same*** end-effector pose;

- For robots which do ***NOT*** have 6 (six) joints and a spherical wrist we can use an iterative numerical (optimization) approach to solving the inverse kinematics;

- We also learned about creating ***paths*** to move the end-effector smoothly between two given points;

Wrapping up…

- ***Joint-space*** paths are simple to compute but in general do ***NOT*** result in straight line paths in Cartesian space which may be ***problematic*** for some applications (MoveJ/MoveAbsJ in RAPID)

- ***Straight line*** paths in Cartesian space can be generated but singularities in the workspace may lead to very high joint rates (e.g., joint velocities) (MoveL in RAPID)

«Homework»

Watch the QUT Robot Academy videos:

- Robotic arms and forward kinematics
- Inverse kinematics and robot motion

Bibliography

[1] Corke, P., **Robotics, *Vision and Control: Fundamental Algorithms in Python***, *Springer International Publishing AG*, 3rd Ed., 2023.

[2] Siciliano, B., Sciavicco, Villani, L. & Oriolo, G., ***Robotics: Modeling, Planning and Control***, *Springer-Verlag London Limited,* 2nd Ed., 2009.

Thank you for your attention !